

COSC2793 — Computational Machine Learning: Assignment 3

Histopathology Image Classification

Student: Alexander Lester (s4234132)

Course: COSC2793 Computational Machine Learning

Date: June 2026

1. Introduction & Task Definition

1.1 Overview

This assignment develops a machine learning pipeline to classify histopathology images of colon cells from the CRCHistoPhenotypes dataset (Sirinukunwattana et al., 2016).

Histopathology involves the microscopic examination of biological tissues to detect abnormalities such as cancer, and automating this classification process has the potential to assist pathologists in diagnosis. The dataset consists of 27×27 RGB images extracted from colon tissue sections of 99 patients, each containing a single cell.

Two supervised classification tasks are addressed:

1. **Task 1 — Binary Classification (isCancerous):** Predict whether a given cell image represents a cancerous cell or not. Both `data_labels_mainData.csv` (60 patients) and `data_labels_extraData.csv` (38 patients) are used, giving a combined dataset of 20,279 images across 98 unique patients.
2. **Task 2 — Multiclass Classification (cellType):** Predict the cell type among four classes: fibroblast, inflammatory, epithelial, or others. Only `data_labels_mainData.csv` is used for this task (9,895 images, 60 patients).

1.2 Performance Objectives

The assignment targets the following minimum macro-averaged F1-scores on unseen test patients:

- **isCancerous:** Macro F1 ≥ 0.90
- **Cell-type:** Macro F1 ≥ 0.60

Macro F1-score is used as the primary metric because it treats all classes equally regardless of their frequency, which is important given the class imbalance present in both tasks.

1.3 Learning Objectives

This project applies the machine learning workflow covered across Weeks 1–12 of COSC2793. The objectives are to design and implement a complete pipeline covering exploratory data analysis, unsupervised learning, data preprocessing, patient-wise splitting, model development (including classical ML and deep learning), hyperparameter tuning, advanced techniques, model comparison, and ethical analysis. Experimental findings are linked to theoretical expectations from course lectures.

1.4 Expected Challenges

Several challenges are anticipated:

- **Small image size (27×27):** Limited spatial information per image restricts the complexity of convolutional architectures.
- **Class imbalance:** Both tasks exhibit imbalanced class distributions, requiring careful metric selection and potential class weighting.
- **Patient-level generalization:** Models must generalize to entirely unseen patients, which is harder than random image-level splitting because different patients may have

2. Imports & Reproducibility

All required libraries are imported below. Seeds are fixed for NumPy, Python's random module, and PyTorch to ensure reproducibility.


```
In [1]: import os
import random
import warnings
import time

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from PIL import Image

from sklearn.model_selection import GroupShuffleSplit
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.metrics import (
    accuracy_score, f1_score, precision_score, recall_score,
    confusion_matrix, classification_report, ConfusionMatrixDisplay,
    roc_curve, auc
)
from sklearn.utils.class_weight import compute_class_weight
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, WeightedRandomSampler
import torchvision.transforms as transforms

warnings.filterwarnings('ignore')

# --- Reproducibility ---
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.backends.mps.is_available():
    torch.mps.manual_seed(SEED)

# Device configuration
if torch.backends.mps.is_available():
    DEVICE = torch.device('mps')
elif torch.cuda.is_available():
    DEVICE = torch.device('cuda')
else:
    DEVICE = torch.device('cpu')
print(f"Using device: {DEVICE}")

# Plotting style
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 100
plt.rcParams['font.size'] = 11
sns.set_style('whitegrid')
```

```
# Paths
DATA_DIR = 'data'
IMAGE_DIR = os.path.join(DATA_DIR, 'images')
FIGURES_DIR = 'figures'
os.makedirs(FIGURES_DIR, exist_ok=True)

print("All imports successful. Environment ready.")
```

Using device: cpu

All imports successful. Environment ready.

3. Load Data

The dataset consists of two CSV label files and a directory of 27×27 RGB PNG images. The main data contains both isCancerous and cellType labels for 60 patients, while the extra data contains only isCancerous labels for an additional 38 patients.

```
In [2]: # Load label files
main_df = pd.read_csv(os.path.join(DATA_DIR, 'data_labels_mainData.csv'))
extra_df = pd.read_csv(os.path.join(DATA_DIR, 'data_labels_extraData.csv'))

print("Main data shape:", main_df.shape)
print("Extra data shape:", extra_df.shape)
print()
print("Main data columns:", list(main_df.columns))
print("Extra data columns:", list(extra_df.columns))
print()
print("Main data head:")
display(main_df.head())
print()
print("Extra data head:")
display(extra_df.head())
```

Main data shape: (9896, 6)

Extra data shape: (10384, 4)

Main data columns: ['InstanceID', 'patientID', 'ImageName', 'cellTypeName', 'cellType', 'isCancerous']

Extra data columns: ['InstanceID', 'patientID', 'ImageName', 'isCancerous']

Main data head:

	InstanceID	patientID	ImageName	cellTypeName	cellType	isCancerous
0	22405	1	22405.png	fibroblast	0	0
1	22406	1	22406.png	fibroblast	0	0
2	22407	1	22407.png	fibroblast	0	0
3	22408	1	22408.png	fibroblast	0	0
4	22409	1	22409.png	fibroblast	0	0

Extra data head:

	InstanceID	patientID	ImageName	isCancerous
0	12681	61	12681.png	0
1	12682	61	12682.png	0
2	12683	61	12683.png	0
3	12684	61	12684.png	0
4	12685	61	12685.png	0

```
In [3]: # Construct combined dataset for Task 1 (isCancerous)
# Use shared columns only
task1_main = main_df[['InstanceID', 'patientID', 'ImageName', 'isCancerous']]
task1_extra = extra_df[['InstanceID', 'patientID', 'ImageName', 'isCancerous']]
task1_df = pd.concat([task1_main, task1_extra], ignore_index=True)

# Task 2 dataset (cellType) – main data only
task2_df = main_df[['InstanceID', 'patientID', 'ImageName', 'cellTypeName']]

print(f"Task 1 (isCancerous) – Combined dataset: {task1_df.shape[0]} samples, {task1_df['patientID'].nunique()} patients")
print(f"Task 2 (cellType) – Main dataset only: {task2_df.shape[0]} samples, {task2_df['patientID'].nunique()} patients")

# Verify no patient overlap between main and extra
main_patients = set(main_df.patientID.unique())
extra_patients = set(extra_df.patientID.unique())
assert len(main_patients & extra_patients) == 0, "Patient overlap detected"
print(f"\nPatient overlap between main and extra: {main_patients & extra_patients}")
print(f"Total unique patients: {len(main_patients | extra_patients)}")
```

Task 1 (isCancerous) – Combined dataset: 20280 samples, 98 patients

Task 2 (cellType) – Main dataset only: 9896 samples, 60 patients

Patient overlap between main and extra: set() (confirmed: no overlap)

Total unique patients: 98

4. Initial Inspection

This section provides a high-level overview of the data quality before any modelling.

```
In [4]: # Missing values
print("=== Missing Values ===")
print("\nMain data:")
print(main_df.isnull().sum())
print("\nExtra data:")
print(extra_df.isnull().sum())

# Duplicates
print("\n=== Duplicate Check ===")
print(f"Duplicate InstanceIDs in main: {main_df.InstanceID.duplicated().sum()}")
print(f"Duplicate InstanceIDs in extra: {extra_df.InstanceID.duplicated().sum()}")
print(f"Overlapping InstanceIDs: {len(set(main_df.InstanceID) & set(extra_df.InstanceID))}")

# Verify all images exist
missing_images = []
for img_name in task1_df.ImageName:
    if not os.path.exists(os.path.join(IMAGE_DIR, img_name)):
        missing_images.append(img_name)
print(f"\nMissing image files: {len(missing_images)}")

# Check image properties
sample_img = Image.open(os.path.join(IMAGE_DIR, task1_df.ImageName.iloc[0]))
sample_arr = np.array(sample_img)
print(f"\n=== Image Properties ===")
print(f"Image size: {sample_img.size}")
print(f"Image mode: {sample_img.mode}")
print(f"Array shape: {sample_arr.shape}")
print(f>Data type: {sample_arr.dtype}")
print(f"Pixel value range: [{sample_arr.min()}, {sample_arr.max()}]")
```


=== Missing Values ===

Main data:

InstanceID 0
patientID 0
ImageName 0
cellTypeName 0
cellType 0
isCancerous 0
dtype: int64

Extra data:

InstanceID 0
patientID 0
ImageName 0
isCancerous 0
dtype: int64

=== Duplicate Check ===

Duplicate InstanceIDs in main: 0
Duplicate InstanceIDs in extra: 0
Overlapping InstanceIDs: 0

Missing image files: 0

=== Image Properties ===

Image size: (27, 27)
Image mode: RGB
Array shape: (27, 27, 3)
Data type: uint8
Pixel value range: [78, 255]

5. Exploratory Data Analysis (EDA)

This section explores the dataset to understand class distributions, patient characteristics, and visual properties of the images. The findings inform preprocessing and modelling decisions.

5.1 Target Distributions

```
In [5]: # Task 1: isCancerous distribution
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# isCancerous distribution
counts_cancer = task1_df.isCancerous.value_counts().sort_index()
labels_cancer = ['Non-Cancerous (0)', 'Cancerous (1)']
colors_cancer = ['#2ecc71', '#e74c3c']
bars = axes[0].bar(labels_cancer, counts_cancer.values, color=colors_cancer)
for bar, count in zip(bars, counts_cancer.values):
    pct = count / len(task1_df) * 100
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 100,
                 f'{count}\n({pct:.1f}%)', ha='center', va='bottom', fontweight='bold')
axes[0].set_title('Task 1: isCancerous Distribution (Combined Data)', fontweight='bold')
axes[0].set_ylabel('Count')
axes[0].set_ylim(0, max(counts_cancer.values) * 1.2)

# cellType distribution
counts_cell = task2_df.cellTypeName.value_counts()
labels_cell = counts_cell.index.tolist()
colors_cell = ['#3498db', '#e67e22', '#9b59b6', '#1abc9c']
bars = axes[1].bar(labels_cell, counts_cell.values, color=colors_cell, edgecolor='black')
for bar, count in zip(bars, counts_cell.values):
    pct = count / len(task2_df) * 100
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 50,
                 f'{count}\n({pct:.1f}%)', ha='center', va='bottom', fontweight='bold')
axes[1].set_title('Task 2: Cell Type Distribution (Main Data)', fontweight='bold')
axes[1].set_ylabel('Count')
axes[1].set_ylim(0, max(counts_cell.values) * 1.2)

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'target_distributions.png'), dpi=100)
plt.show()
print("Figure 1: Target variable distributions for both tasks.")
```

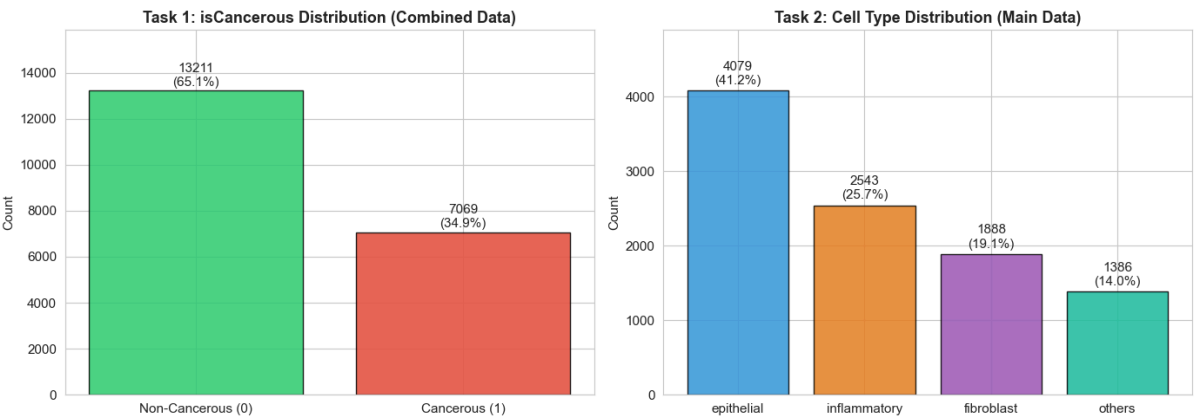


Figure 1: Target variable distributions for both tasks.

Interpretation: The isCancerous task shows moderate class imbalance with non-cancerous cells (65.1%) outnumbering cancerous cells (34.9%). The cell-type task has a more pronounced imbalance: epithelial cells dominate (41.2%), followed by inflammatory (25.7%), fibroblast (19.1%), and others (14.0%). This imbalance means that a naive classifier could achieve reasonable accuracy by predicting the majority class, but would perform poorly on minority classes. Macro F1-score is therefore the appropriate primary metric, as it weights all classes equally regardless of frequency. Class weighting in model training will be used to help the models learn from underrepresented classes.

5.2 Patient Distribution

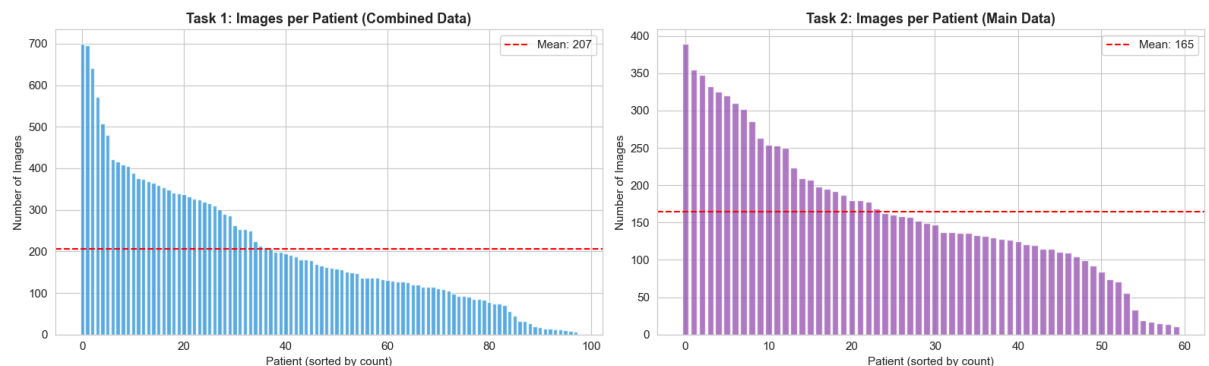
```
In [6]: # Images per patient
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Task 1: images per patient
patient_counts_t1 = task1_df.groupby('patientID').size().sort_values(asc
axes[0].bar(range(len(patient_counts_t1)), patient_counts_t1.values, col
axes[0].set_title('Task 1: Images per Patient (Combined Data)', fontsize=
axes[0].set_xlabel('Patient (sorted by count)')
axes[0].set_ylabel('Number of Images')
axes[0].axhline(y=patient_counts_t1.mean(), color='red', linestyle='--',
axes[0].legend()

# Task 2: images per patient
patient_counts_t2 = task2_df.groupby('patientID').size().sort_values(asc
axes[1].bar(range(len(patient_counts_t2)), patient_counts_t2.values, col
axes[1].set_title('Task 2: Images per Patient (Main Data)', fontsize=13,
axes[1].set_xlabel('Patient (sorted by count)')
axes[1].set_ylabel('Number of Images')
axes[1].axhline(y=patient_counts_t2.mean(), color='red', linestyle='--',
axes[1].legend()

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'patient_distribution.png'), dpi=1
plt.show()

print(f"Task 1 - Min images per patient: {patient_counts_t1.min()}, Max:
print(f"Task 2 - Min images per patient: {patient_counts_t2.min()}, Max:
```



Task 1 - Min images per patient: 6, Max: 699, Mean: 206.9

Task 2 - Min images per patient: 11, Max: 389, Mean: 164.9

Interpretation: There is considerable variation in the number of images per patient. Some patients contribute hundreds of images while others contribute fewer than 50. This variability means that patient-wise splitting is critical: if images from a single prolific patient appeared in both training and test sets, the model could learn patient-specific tissue patterns rather than generalisable cell-type features.

5.3 Image Samples by Class

```
In [7]: # Visualise sample images for each class
def show_class_samples(df, label_col, class_names, title, n_samples=5):
    """Display n_samples random images for each class."""
    classes = sorted(df[label_col].unique())
    fig, axes = plt.subplots(len(classes), n_samples, figsize=(n_samples
fig.suptitle(title, fontsize=14, fontweight='bold', y=1.02)

    for i, cls in enumerate(classes):
        cls_df = df[df[label_col] == cls]
        samples = cls_df.sample(n=min(n_samples, len(cls_df)), random_st
        for j, (_, row) in enumerate(samples.iterrows()):
            img = Image.open(os.path.join(IMAGE_DIR, row['ImageName']))
            axes[i, j].imshow(img)
            axes[i, j].axis('off')
            if j == 0:
                if label_col == 'isCancerous':
                    name = 'Non-Cancerous' if cls == 0 else 'Cancerous'
                else:
                    name = class_names[cls] if isinstance(cls, int) else
            axes[i, j].set_ylabel(name, fontsize=11, fontweight='bol

    plt.tight_layout()
    return fig

# Task 1: isCancerous samples
fig = show_class_samples(task1_df, 'isCancerous', {0: 'Non-Cancerous', 1
                        'Task 1: Sample Images – Cancerous vs Non-Cance
plt.savefig(os.path.join(FIGURES_DIR, 'samples_isCancerous.png'), dpi=15
plt.show()
print("Figure 3: Random image samples for isCancerous classes.")

# Task 2: cellType samples
fig = show_class_samples(task2_df, 'cellTypeName', None,
                        'Task 2: Sample Images – Cell Types', n_samples
plt.savefig(os.path.join(FIGURES_DIR, 'samples_cellType.png'), dpi=150,
plt.show()
print("Figure 4: Random image samples for cell type classes.")
```

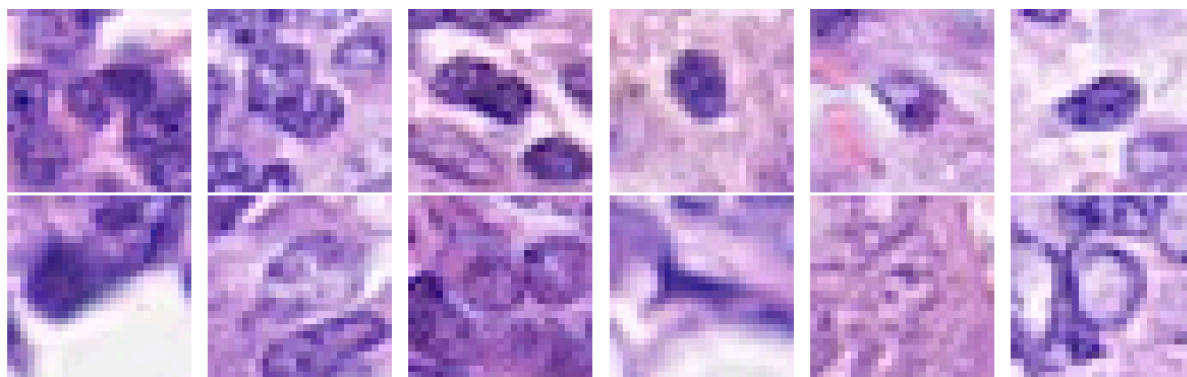
Task 1: Sample Images — Cancerous vs Non-Cancerous

Figure 3: Random image samples for isCancerous classes.

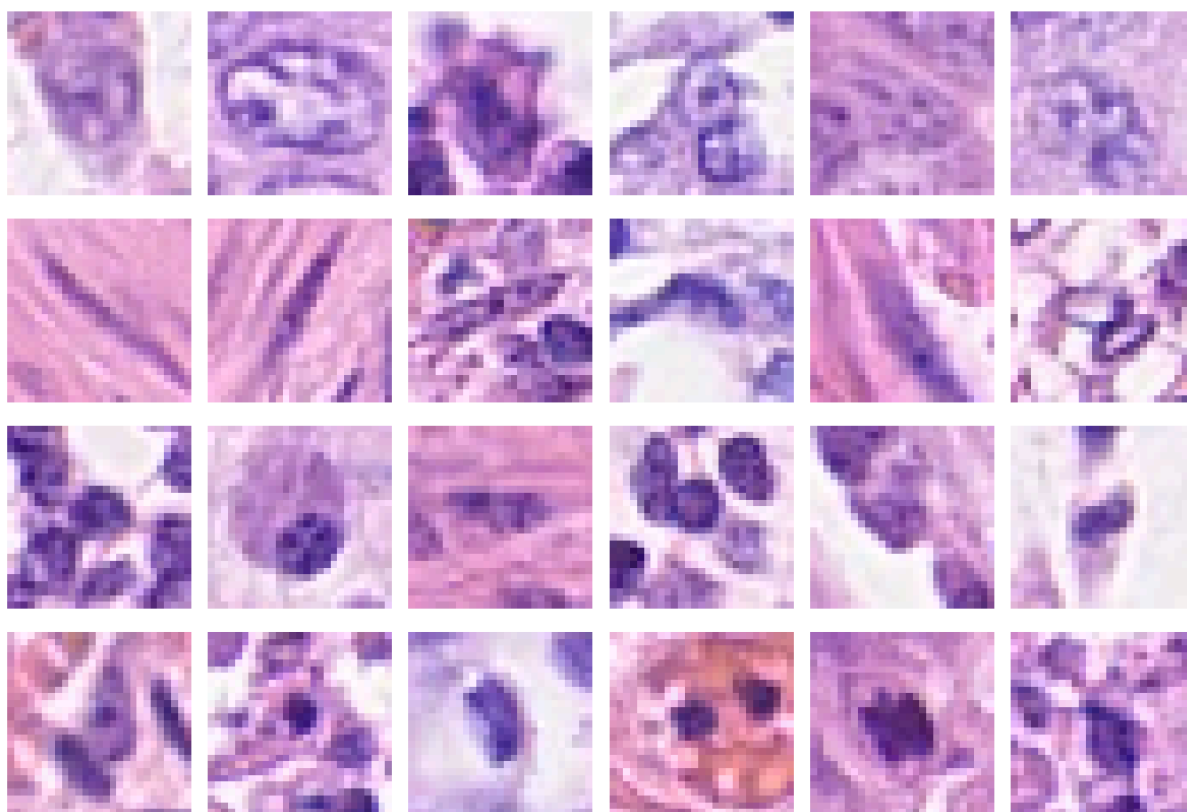
Task 2: Sample Images — Cell Types

Figure 4: Random image samples for cell type classes.

Interpretation: Visual inspection reveals that the images are very small (27×27 pixels) and show cellular structures stained with haematoxylin and eosin (H&E). The difference between cancerous and non-cancerous cells is not always visually obvious at this resolution, though cancerous cells (epithelial) tend to show more intense staining. The four cell types show subtle morphological differences: epithelial cells tend to be more densely stained, inflammatory cells are smaller and darker, fibroblasts are elongated, and the "others" category is heterogeneous. These visual similarities suggest that the classification task is non-trivial, particularly for the multiclass problem.

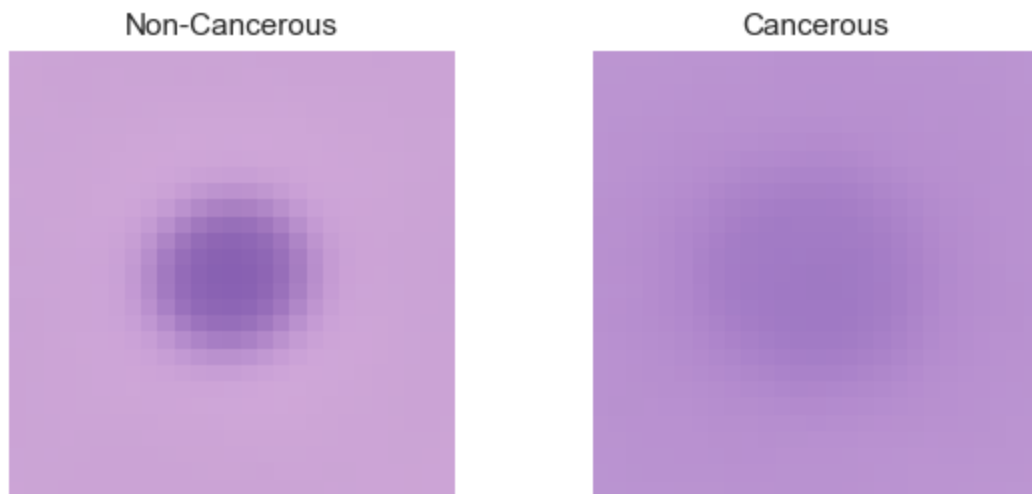
5.4 Mean Images by Class

```
In [8]: # Compute and display mean images per class
def compute_mean_image(df, label_col, image_dir):
    """Compute the mean image for each class."""
    classes = sorted(df[label_col].unique())
    mean_images = {}
    for cls in classes:
        cls_df = df[df[label_col] == cls]
        imgs = []
        for _, row in cls_df.iterrows():
            img = np.array(Image.open(os.path.join(image_dir, row['Image'])))
            imgs.append(img.astype(np.float64))
        mean_images[cls] = np.mean(imgs, axis=0).astype(np.uint8)
    return mean_images

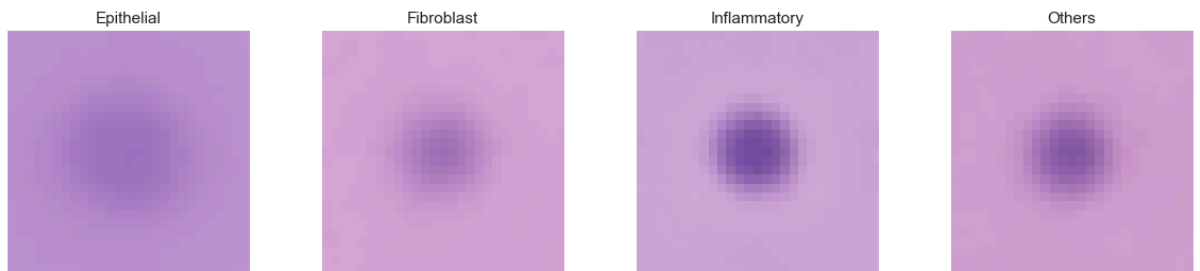
# Task 1 mean images
mean_cancer = compute_mean_image(task1_df, 'isCancerous', IMAGE_DIR)
fig, axes = plt.subplots(1, 2, figsize=(6, 3))
fig.suptitle('Task 1: Mean Images by isCancerous', fontsize=13, fontweight='bold')
for i, (cls, label) in enumerate([(0, 'Non-Cancerous'), (1, 'Cancerous')]):
    axes[i].imshow(mean_cancer[cls])
    axes[i].set_title(label, fontsize=11)
    axes[i].axis('off')
plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'mean_images_cancer.png'), dpi=150)
plt.show()

# Task 2 mean images
mean_cell = compute_mean_image(task2_df, 'cellTypeName', IMAGE_DIR)
fig, axes = plt.subplots(1, 4, figsize=(12, 3))
fig.suptitle('Task 2: Mean Images by Cell Type', fontsize=13, fontweight='bold')
cell_names = ['epithelial', 'fibroblast', 'inflammatory', 'others']
for i, name in enumerate(cell_names):
    if name in mean_cell:
        axes[i].imshow(mean_cell[name])
        axes[i].set_title(name.capitalize(), fontsize=11)
        axes[i].axis('off')
plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'mean_images_celltype.png'), dpi=150)
plt.show()
print("Figures 5–6: Mean images by class reveal structural tendencies across classes.")
```

Task 1: Mean Images by isCancerous



Task 2: Mean Images by Cell Type



Figures 5–6: Mean images by class reveal structural tendencies across classes.

Interpretation: The mean images show that cancerous cells tend to have a more concentrated, darker central region compared to non-cancerous cells. Among cell types, epithelial cells show the most intense central staining, while fibroblasts and the "others" category appear lighter on average. These differences are subtle but measurable, supporting the feasibility of automated classification. However, the blurriness of the mean images confirms that individual images vary significantly within each class, meaning the models will need to capture more complex patterns than simple average pixel values.

5.5 Pixel Intensity Analysis

```
In [9]: # Pixel intensity statistics per class
def pixel_stats_by_class(df, label_col, image_dir, n_samples=2000):
    """Compute pixel intensity statistics per class."""
    classes = sorted(df[label_col].unique())
    stats = {}
    for cls in classes:
        cls_df = df[df[label_col] == cls]
        if len(cls_df) > n_samples:
            cls_df = cls_df.sample(n=n_samples, random_state=SEED)
        pixels = []
        for _, row in cls_df.iterrows():
            img = np.array(Image.open(os.path.join(image_dir, row['ImageName'])))
            pixels.append(img.reshape(-1, 3).mean(axis=0)) # mean RGB pixel
        pixels = np.array(pixels)
        stats[cls] = {
            'R_mean': pixels[:, 0].mean(), 'G_mean': pixels[:, 1].mean(),
            'R_std': pixels[:, 0].std(), 'G_std': pixels[:, 1].std(), 'B_std': pixels[:, 2].std()
        }
    return pd.DataFrame(stats).T

print("=== Task 1: Pixel Statistics by isCancerous ===")
stats_t1 = pixel_stats_by_class(task1_df, 'isCancerous', IMAGE_DIR)
stats_t1.index = ['Non-Cancerous', 'Cancerous']
display(stats_t1.round(2))

print("\n=== Task 2: Pixel Statistics by Cell Type ===")
stats_t2 = pixel_stats_by_class(task2_df, 'cellTypeName', IMAGE_DIR)
display(stats_t2.round(2))
```

=== Task 1: Pixel Statistics by isCancerous ===

	R_mean	G_mean	B_mean	R_std	G_std	B_std
Non-Cancerous	199.23	159.46	211.91	15.45	21.03	11.89
Cancerous	180.63	140.90	206.21	15.57	21.34	12.98

=== Task 2: Pixel Statistics by Cell Type ===

	R_mean	G_mean	B_mean	R_std	G_std	B_std
epithelial	181.21	138.47	202.82	17.10	23.27	14.47
fibroblast	207.62	159.32	208.36	10.53	20.55	12.93
inflammatory	198.07	159.84	208.29	17.23	22.82	12.95
others	201.56	152.77	202.24	17.98	25.56	16.25

Interpretation: The pixel intensity statistics show measurable differences between classes across all three colour channels. Cancerous cells tend to have slightly lower mean intensity (darker staining) compared to non-cancerous cells, consistent with their more active cellular state. Among cell types, epithelial cells show the lowest mean pixel values (darkest staining), while fibroblasts and others show higher mean intensities. These channel-level differences, while modest, provide useful signal for pixel-based classifiers.

6. Unsupervised Learning — Dimensionality Reduction

To further understand the structure of the data before modelling, PCA and t-SNE are applied as unsupervised techniques. As covered in Lecture 11 (Dimensionality Reduction), PCA finds orthogonal principal components that capture the most variance in the data, while t-SNE provides a nonlinear embedding for visualisation in 2D. By applying these techniques to flattened image vectors, we can assess whether the classes naturally form separable clusters, which informs expectations about model complexity.

6.1 PCA — Explained Variance

```

In [10]: # Load a subset of images for unsupervised analysis
def load_images_flat(df, image_dir, max_samples=5000):
    """Load images and flatten to 1D vectors."""
    if len(df) > max_samples:
        df_sub = df.sample(n=max_samples, random_state=SEED)
    else:
        df_sub = df

    images = []
    for _, row in df_sub.iterrows():
        img = np.array(Image.open(os.path.join(image_dir, row['ImageName'])))
        images.append(img.flatten().astype(np.float32) / 255.0)

    return np.array(images), df_sub

# Task 1 images
X_flat_t1, df_sub_t1 = load_images_flat(task1_df, IMAGE_DIR, max_samples=5000)
print(f"Task 1 subset: {X_flat_t1.shape[0]} images, {X_flat_t1.shape[1]} features")

# PCA
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_flat_t1)

pca_full = PCA(random_state=SEED)
pca_full.fit(X_scaled)

# Explained variance plot
cumvar = np.cumsum(pca_full.explained_variance_ratio_)
n_95 = np.argmax(cumvar >= 0.95) + 1
n_90 = np.argmax(cumvar >= 0.90) + 1

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(range(1, len(pca_full.explained_variance_ratio_) + 1),
             pca_full.explained_variance_ratio_, 'b-', alpha=0.7)
axes[0].set_xlabel('Principal Component')
axes[0].set_ylabel('Explained Variance Ratio')
axes[0].set_title('Individual Explained Variance', fontsize=13, fontweight='bold')
axes[0].set_xlim(0, 100)

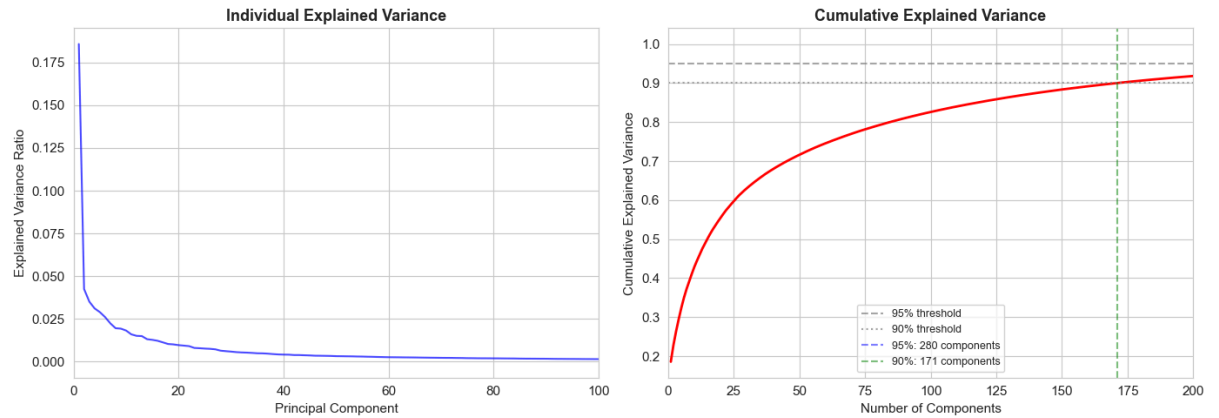
axes[1].plot(range(1, len(cumvar) + 1), cumvar, 'r-', linewidth=2)
axes[1].axhline(y=0.95, color='gray', linestyle='--', alpha=0.7, label='95%')
axes[1].axhline(y=0.90, color='gray', linestyle=':', alpha=0.7, label='90%')
axes[1].axvline(x=n_95, color='blue', linestyle='--', alpha=0.5, label=f'95% at {n_95}')
axes[1].axvline(x=n_90, color='green', linestyle='--', alpha=0.5, label=f'90% at {n_90}')
axes[1].set_xlabel('Number of Components')
axes[1].set_ylabel('Cumulative Explained Variance')
axes[1].set_title('Cumulative Explained Variance', fontsize=13, fontweight='bold')
axes[1].legend(fontsize=9)
axes[1].set_xlim(0, 200)

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'pca_explained_variance.png'), dpi=100)
plt.show()
print(f"Components for 90% variance: {n_90}")
print(f"Components for 95% variance: {n_95}")

```

```
print(f"Total features: {X_flat_t1.shape[1]}")
```

Task 1 subset: 5000 images, 2187 features (flattened)



Components for 90% variance: 171
Components for 95% variance: 280
Total features: 2187

Interpretation: PCA reveals that the first few principal components capture a disproportionate share of the total variance, which is typical for image data where many pixels are correlated. Retaining 95% of the variance requires significantly fewer components than the original 2,187 features, confirming that the intrinsic dimensionality of the data is much lower than the raw pixel space. This is encouraging for classical ML models, which can benefit from dimensionality reduction to avoid the curse of dimensionality.

6.2 t-SNE Visualisation

```

In [11]: # Reduce to 50 components first (for t-SNE speed)
pca_50 = PCA(n_components=50, random_state=SEED)
X_pca50 = pca_50.fit_transform(X_scaled)

# t-SNE
tsne = TSNE(n_components=2, perplexity=30, random_state=SEED, max_iter=1000)
X_tsne = tsne.fit_transform(X_pca50)

# Plot t-SNE coloured by isCancerous
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

scatter1 = axes[0].scatter(X_tsne[:, 0], X_tsne[:, 1],
                           c=df_sub_t1.isCancerous.values, cmap='RdYlGn',
                           alpha=0.4, s=8, edgecolors='none')
axes[0].set_title('t-SNE coloured by isCancerous', fontsize=13, fontweight='bold')
axes[0].set_xlabel('t-SNE 1')
axes[0].set_ylabel('t-SNE 2')
legend1 = axes[0].legend(*scatter1.legend_elements(), title='isCancerous')

# For Task 2, load subset of main data
X_flat_t2, df_sub_t2 = load_images_flat(task2_df, IMAGE_DIR, max_samples=1000)
X_scaled_t2 = StandardScaler().fit_transform(X_flat_t2)
X_pca50_t2 = PCA(n_components=50, random_state=SEED).fit_transform(X_scaled_t2)
X_tsne_t2 = TSNE(n_components=2, perplexity=30, random_state=SEED, max_iter=1000).fit_transform(X_pca50_t2)

# Encode cell type names to integers for colouring
cell_type_map = {'fibroblast': 0, 'inflammatory': 1, 'epithelial': 2, 'others': 3}
cell_colors = df_sub_t2.cellTypeName.map(cell_type_map).values

scatter2 = axes[1].scatter(X_tsne_t2[:, 0], X_tsne_t2[:, 1],
                           c=cell_colors, cmap='tab10',
                           alpha=0.4, s=8, edgecolors='none')
axes[1].set_title('t-SNE coloured by Cell Type', fontsize=13, fontweight='bold')
axes[1].set_xlabel('t-SNE 1')
axes[1].set_ylabel('t-SNE 2')
cell_labels = ['fibroblast', 'inflammatory', 'epithelial', 'others']
handles = [plt.Line2D([0], [0], marker='o', color='w', markerfacecolor=p,
                      markersize=8) for i in range(4)]
axes[1].legend(handles, cell_labels, title='Cell Type', loc='upper right')

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'tsne_visualisation.png'), dpi=150)
plt.show()
print("Figure 8: t-SNE 2D projections coloured by class labels.")

```

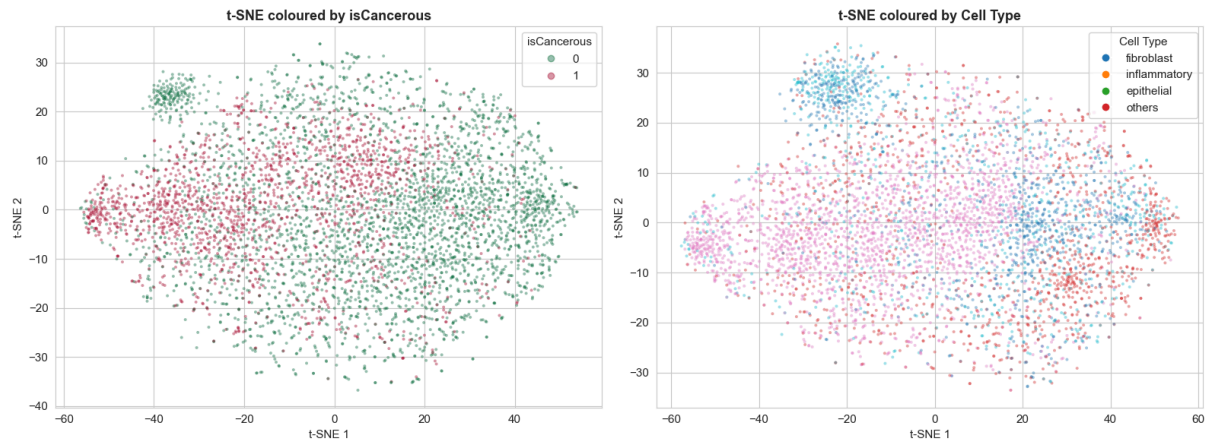


Figure 8: t-SNE 2D projections coloured by class labels.

Interpretation: The t-SNE visualisation reveals partial but imperfect class separation. For the `isCancerous` task, cancerous and non-cancerous cells form some distinct clusters, but there is substantial overlap in the embedding space. This suggests that a binary classifier can achieve reasonable performance, but perfect separation from pixel data alone is unlikely. For the cell-type task, the four classes show even more overlap, with epithelial and inflammatory cells particularly intermixed. This is consistent with the expected difficulty of multiclass histopathology classification and suggests that achieving a Macro F1 above 0.60 for cell-type classification will require carefully tuned models. The overlap also explains why more complex models (such as CNNs that can learn spatial features) may outperform linear classifiers that operate on flattened pixel vectors.

7. Data Splitting — Patient-Wise

7.1 Splitting Strategy

The assignment explicitly requires that the model generalises to **new unseen patients**. This means the data must be split at the patient level, not the image level. If images from the same patient appeared in both the training and test sets, the model could exploit patient-specific tissue characteristics (such as staining intensity or tissue quality) rather than learning generalisable cell-type features. This would inflate test performance and give a misleading estimate of real-world generalisation.

A **60/20/20 train/validation/test split** is used, with `GroupShuffleSplit` from scikit-learn to ensure that all images from a given patient appear in exactly one subset. The validation set is used for hyperparameter tuning and model selection during development, while the test set is reserved for final evaluation only.


```

In [12]: def patient_wise_split(df, test_size=0.20, val_size=0.20, random_state=S
        """
        Split data into train/validation/test sets by patient ID.
        Ensures zero patient overlap between any two sets.
        """

        # First split: separate test set (20%)
        gss_test = GroupShuffleSplit(n_splits=1, test_size=test_size, random
        train_val_idx, test_idx = next(gss_test.split(df, groups=df['patient

        df_train_val = df.iloc[train_val_idx].reset_index(drop=True)
        df_test = df.iloc[test_idx].reset_index(drop=True)

        # Second split: separate validation set (20% of original = 25% of re
        val_fraction = val_size / (1 - test_size)
        gss_val = GroupShuffleSplit(n_splits=1, test_size=val_fraction, rand
        train_idx, val_idx = next(gss_val.split(df_train_val, groups=df_trai

        df_train = df_train_val.iloc[train_idx].reset_index(drop=True)
        df_val = df_train_val.iloc[val_idx].reset_index(drop=True)

        return df_train, df_val, df_test

# Split Task 1
train_t1, val_t1, test_t1 = patient_wise_split(task1_df)

# Split Task 2
train_t2, val_t2, test_t2 = patient_wise_split(task2_df)

# Verification
for name, (tr, va, te), df_full in [
    ("Task 1", (train_t1, val_t1, test_t1), task1_df),
    ("Task 2", (train_t2, val_t2, test_t2), task2_df)
]:
    tr_patients = set(tr.patientID.unique())
    va_patients = set(va.patientID.unique())
    te_patients = set(te.patientID.unique())

    print(f"\n{'='*50}")
    print(f"{name} Split Summary")
    print(f"{'='*50}")
    print(f"  Train:      {len(tr):>6} samples, {len(tr_patients):>3} pa
    print(f"  Validation: {len(va):>6} samples, {len(va_patients):>3} pa
    print(f"  Test:      {len(te):>6} samples, {len(te_patients):>3} pa
    print(f"  Total:      {len(tr)+len(va)+len(te):>6} / {len(df_full)}")
    print(f"\n  Train patients:      {sorted(tr_patients)}")
    print(f"  Validation patients: {sorted(va_patients)}")
    print(f"  Test patients:      {sorted(te_patients)}")

# Assert no overlap
assert len(tr_patients & va_patients) == 0, f"{name}: Train-Val pati
assert len(tr_patients & te_patients) == 0, f"{name}: Train-Test pat
assert len(va_patients & te_patients) == 0, f"{name}: Val-Test patie
print(f"\n ✓ No patient overlap between any splits confirmed.")

```

```
=====
Task 1 Split Summary
=====
```

```
Train:      12416 samples,  58 patients
Validation:  3908 samples,  20 patients
Test:       3956 samples,  20 patients
Total:      20280 / 20280
```

```
Train patients: [np.int64(3), np.int64(4), np.int64(6), np.int64(
9), np.int64(10), np.int64(12), np.int64(16), np.int64(18), np.int64(2
0), np.int64(21), np.int64(22), np.int64(24), np.int64(26), np.int64(2
8), np.int64(29), np.int64(31), np.int64(33), np.int64(34), np.int64(3
5), np.int64(36), np.int64(37), np.int64(38), np.int64(42), np.int64(4
7), np.int64(49), np.int64(50), np.int64(51), np.int64(52), np.int64(5
3), np.int64(54), np.int64(55), np.int64(56), np.int64(58), np.int64(5
9), np.int64(60), np.int64(61), np.int64(64), np.int64(66), np.int64(6
7), np.int64(68), np.int64(70), np.int64(71), np.int64(72), np.int64(7
5), np.int64(78), np.int64(80), np.int64(81), np.int64(82), np.int64(8
4), np.int64(87), np.int64(88), np.int64(89), np.int64(90), np.int64(9
1), np.int64(94), np.int64(95), np.int64(97), np.int64(98)]
```

```
Validation patients: [np.int64(2), np.int64(7), np.int64(8), np.int64
(14), np.int64(15), np.int64(17), np.int64(25), np.int64(30), np.int64
(39), np.int64(40), np.int64(44), np.int64(46), np.int64(57), np.int64
(62), np.int64(69), np.int64(73), np.int64(79), np.int64(86), np.int64
(93), np.int64(99)]
```

```
Test patients: [np.int64(1), np.int64(5), np.int64(11), np.int64(1
3), np.int64(19), np.int64(23), np.int64(27), np.int64(32), np.int64(
41), np.int64(43), np.int64(45), np.int64(48), np.int64(63), np.int64
(65), np.int64(74), np.int64(77), np.int64(83), np.int64(85), np.int64
(92), np.int64(96)]
```

✓ No patient overlap between any splits confirmed.

```
=====
Task 2 Split Summary
=====
```

```
Train:      5671 samples,  36 patients
Validation:  2386 samples,  12 patients
Test:       1839 samples,  12 patients
Total:      9896 / 9896
```

```
Train patients: [np.int64(2), np.int64(3), np.int64(4), np.int64(
8), np.int64(9), np.int64(10), np.int64(12), np.int64(15), np.int64(1
6), np.int64(18), np.int64(19), np.int64(20), np.int64(21), np.int64(2
2), np.int64(23), np.int64(25), np.int64(26), np.int64(27), np.int64(2
8), np.int64(33), np.int64(35), np.int64(36), np.int64(38), np.int64(3
9), np.int64(40), np.int64(41), np.int64(42), np.int64(43), np.int64(4
5), np.int64(48), np.int64(52), np.int64(53), np.int64(56), np.int64(5
7), np.int64(59), np.int64(60)]
```

```
Validation patients: [np.int64(5), np.int64(7), np.int64(11), np.int64
(17), np.int64(24), np.int64(29), np.int64(30), np.int64(31), np.int64
(32), np.int64(44), np.int64(50), np.int64(54)]
```

```
Test patients: [np.int64(1), np.int64(6), np.int64(13), np.int64(1
4), np.int64(34), np.int64(37), np.int64(46), np.int64(47), np.int64
(49), np.int64(51), np.int64(55), np.int64(58)]
```

✓ No patient overlap between any splits confirmed.

Verification: The output confirms zero patient overlap between train, validation, and test sets for both tasks. Approximately 60% of images are in the training set, 20% in validation, and 20% in test, with the exact proportions depending on patient sizes. This patient-wise splitting approach is critical for obtaining a realistic estimate of model performance on new, unseen patients.

8. Data Preprocessing

Two preprocessing pipelines are constructed:

1. **Flat pipeline** (for classical ML models): Images are loaded, flattened to 1D vectors (2,187 features), and standardised using `StandardScaler` fitted on the training set only.
2. **Tensor pipeline** (for CNN): Images are loaded as 3D tensors, normalised to $[0, 1]$, and served via `PyTorch DataLoader`.

Fitting the scaler only on training data prevents information from validation or test sets leaking into the training process, consistent with proper machine learning practice (Lecture 4).

```

In [13]: # ===== Flat preprocessing for classical ML =====
def load_flat_images(df, image_dir):
    """Load images, flatten to 1D, return array."""
    images = []
    for _, row in df.iterrows():
        img = np.array(Image.open(os.path.join(image_dir, row['ImageName'])))
        images.append(img.flatten().astype(np.float32))
    return np.array(images)

print("Loading flattened images for classical ML...")
t = time.time()

# Task 1
X_train_flat_t1 = load_flat_images(train_t1, IMAGE_DIR)
X_val_flat_t1 = load_flat_images(val_t1, IMAGE_DIR)
X_test_flat_t1 = load_flat_images(test_t1, IMAGE_DIR)
y_train_t1 = train_t1['isCancerous'].values
y_val_t1 = val_t1['isCancerous'].values
y_test_t1 = test_t1['isCancerous'].values

# Task 2
X_train_flat_t2 = load_flat_images(train_t2, IMAGE_DIR)
X_val_flat_t2 = load_flat_images(val_t2, IMAGE_DIR)
X_test_flat_t2 = load_flat_images(test_t2, IMAGE_DIR)
y_train_t2 = train_t2['cellType'].values
y_val_t2 = val_t2['cellType'].values
y_test_t2 = test_t2['cellType'].values

print(f"Loaded in {time.time()-t:.1f}s")

# Standardise using training data only
scaler_t1 = StandardScaler()
X_train_scaled_t1 = scaler_t1.fit_transform(X_train_flat_t1)
X_val_scaled_t1 = scaler_t1.transform(X_val_flat_t1)
X_test_scaled_t1 = scaler_t1.transform(X_test_flat_t1)

scaler_t2 = StandardScaler()
X_train_scaled_t2 = scaler_t2.fit_transform(X_train_flat_t2)
X_val_scaled_t2 = scaler_t2.transform(X_val_flat_t2)
X_test_scaled_t2 = scaler_t2.transform(X_test_flat_t2)

print(f"\nTask 1 - Train: {X_train_scaled_t1.shape}, Val: {X_val_scaled_t1.shape}, Test: {X_test_scaled_t1.shape}")
print(f"Task 2 - Train: {X_train_scaled_t2.shape}, Val: {X_val_scaled_t2.shape}, Test: {X_test_scaled_t2.shape}")

```

Loading flattened images for classical ML...

Loaded in 4.6s

Task 1 - Train: (12416, 2187), Val: (3908, 2187), Test: (3956, 2187)

Task 2 - Train: (5671, 2187), Val: (2386, 2187), Test: (1839, 2187)


```

In [14]: # ===== PyTorch Dataset & DataLoader for CNN =====
class CellImageDataset(Dataset):
    """Custom PyTorch Dataset for cell images."""
    def __init__(self, df, image_dir, label_col, transform=None):
        self.df = df.reset_index(drop=True)
        self.image_dir = image_dir
        self.label_col = label_col
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img = Image.open(os.path.join(self.image_dir, row['ImageName']))
        img = np.array(img).astype(np.float32) / 255.0 # normalise to [0,1]
        img = torch.tensor(img).permute(2, 0, 1) # (H,W,C) -> (C,H,W)

        if self.transform:
            img = self.transform(img)

        label = torch.tensor(row[self.label_col], dtype=torch.long)
        return img, label

# --- WeightedRandomSampler for class-balanced training batches ---
# WeightedRandomSampler ensures each mini-batch contains a balanced repr
# of all classes. This complements weighted CrossEntropyLoss: the sample
# how often each class is SEEN during training, while the loss weights c
# much each class CONTRIBUTES to the gradient. Together, they substantia
# minority class learning, which is critical for Macro F1 on the cellTyp
BATCH_SIZE = 64

def make_weighted_sampler(labels):
    # Create a WeightedRandomSampler that oversamples minority classes
    class_counts = np.bincount(labels)
    class_weights = 1.0 / class_counts.astype(np.float64)
    sample_weights = class_weights[labels]
    sampler = WeightedRandomSampler(
        weights=torch.from_numpy(sample_weights).float(),
        num_samples=len(labels), replacement=True
    )
    return sampler

sampler_t1 = make_weighted_sampler(y_train_t1)
sampler_t2 = make_weighted_sampler(y_train_t2)
print("WeightedRandomSampler created for both tasks.")

# Task 1 DataLoaders (sampler on train only - val/test use no sampling)
train_ds_t1 = CellImageDataset(train_t1, IMAGE_DIR, 'isCancerous')
val_ds_t1 = CellImageDataset(val_t1, IMAGE_DIR, 'isCancerous')
test_ds_t1 = CellImageDataset(test_t1, IMAGE_DIR, 'isCancerous')

train_loader_t1 = DataLoader(train_ds_t1, batch_size=BATCH_SIZE, sampler=sampler_t1)
val_loader_t1 = DataLoader(val_ds_t1, batch_size=BATCH_SIZE, shuffle=False)
test_loader_t1 = DataLoader(test_ds_t1, batch_size=BATCH_SIZE, shuffle=False)

# Task 2 DataLoaders (sampler on train only - val/test use no sampling)

```

```

train_ds_t2 = CellImageDataset(train_t2, IMAGE_DIR, 'cellType')
val_ds_t2 = CellImageDataset(val_t2, IMAGE_DIR, 'cellType')
test_ds_t2 = CellImageDataset(test_t2, IMAGE_DIR, 'cellType')

train_loader_t2 = DataLoader(train_ds_t2, batch_size=BATCH_SIZE, sampler=None)
val_loader_t2 = DataLoader(val_ds_t2, batch_size=BATCH_SIZE, shuffle=False)
test_loader_t2 = DataLoader(test_ds_t2, batch_size=BATCH_SIZE, shuffle=False)

# Verify a batch
images, labels = next(iter(train_loader_t1))
print(f"Batch shape: {images.shape}, Labels shape: {labels.shape}")
print(f"Image value range: [{images.min():.3f}, {images.max():.3f}]")

# Compute class weights for loss functions
def get_class_weights(y, device):
    classes = np.unique(y)
    weights = compute_class_weight('balanced', classes=classes, y=y)
    return torch.tensor(weights, dtype=torch.float32).to(device)

weights_t1 = get_class_weights(y_train_t1, DEVICE)
weights_t2 = get_class_weights(y_train_t2, DEVICE)
print(f"\nTask 1 class weights: {weights_t1}")
print(f"Task 2 class weights: {weights_t2}")

```

WeightedRandomSampler created for both tasks.

Batch shape: torch.Size([64, 3, 27, 27]), Labels shape: torch.Size([64])

Image value range: [0.035, 1.000]

Task 1 class weights: tensor([0.7883, 1.3671])

Task 2 class weights: tensor([1.0708, 1.0091, 0.6375, 1.9746])

9. Evaluation Framework

To compare all models fairly, a consistent evaluation framework is defined. Every model is evaluated using the same metrics on the same train/validation/test splits. As discussed in Lecture 4, using consistent evaluation across models is essential for meaningful comparison.

Metrics used:

- **Accuracy:** Overall proportion of correct predictions. Useful as a general measure but can be misleading under class imbalance.
- **Macro F1-score:** The primary metric. Averages F1-score across all classes equally, ensuring that minority classes are weighted the same as majority classes.
- **Precision & Recall:** Complement F1-score by showing the trade-off between false positives and false negatives.
- **Confusion Matrix:** Visualises which classes are being confused with each other.
- **ROC Curve / AUC:** For the binary isCancerous task, provides a threshold-independent view of classifier performance.

```

In [15]: def evaluate_model(y_true, y_pred, model_name, task_name, class_names=None):
    """Evaluate a model and return a dictionary of metrics."""
    acc = accuracy_score(y_true, y_pred)
    f1_macro = f1_score(y_true, y_pred, average='macro')
    prec_macro = precision_score(y_true, y_pred, average='macro', zero_divis
    rec_macro = recall_score(y_true, y_pred, average='macro', zero_divis

    metrics = {
        'Model': model_name,
        'Task': task_name,
        'Accuracy': acc,
        'Macro F1': f1_macro,
        'Macro Precision': prec_macro,
        'Macro Recall': rec_macro,
    }
    return metrics

def plot_confusion_matrix(y_true, y_pred, class_names, title, save_path=None):
    """Plot a confusion matrix."""
    cm = confusion_matrix(y_true, y_pred)
    fig, ax = plt.subplots(figsize=(6, 5))
    disp = ConfusionMatrixDisplay(cm, display_labels=class_names)
    disp.plot(ax=ax, cmap='Blues', values_format='d')
    ax.set_title(title, fontsize=13, fontweight='bold')
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=150, bbox_inches='tight')
    plt.show()

def plot_roc_curve(y_true, y_score, title, save_path=None):
    """Plot ROC curve for binary classification."""
    fpr, tpr, _ = roc_curve(y_true, y_score)
    roc_auc = auc(fpr, tpr)
    fig, ax = plt.subplots(figsize=(6, 5))
    ax.plot(fpr, tpr, color='#e74c3c', lw=2, label=f'ROC curve (AUC = {r
    ax.plot([0, 1], [0, 1], color='gray', lw=1, linestyle='--')
    ax.set_xlabel('False Positive Rate')
    ax.set_ylabel('True Positive Rate')
    ax.set_title(title, fontsize=13, fontweight='bold')
    ax.legend(loc='lower right')
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=150, bbox_inches='tight')
    plt.show()
    return roc_auc

# Storage for all results
all_results = []
print("Evaluation framework defined.")

```

Evaluation framework defined.

10. Model Development — Baseline Models

This section implements four supervised learning models for each task: Logistic Regression, Support Vector Machine (SVM), Random Forest, and a Convolutional Neural Network (CNN). For each model, the rationale, theoretical basis, and expected strengths/weaknesses are discussed before presenting results.

10.1 Logistic Regression

Model Rationale

Logistic Regression is used as the first baseline because it is a simple, well-understood linear classifier. As covered in Lectures 1–3, logistic regression models the probability of class membership using a sigmoid (binary) or softmax (multiclass) function applied to a linear combination of features. It is computationally efficient and provides a meaningful lower bound on performance: if logistic regression achieves reasonable results, it confirms that there is learnable signal in the pixel features. Its main limitation is that it can only learn linear decision boundaries in feature space, which may not be sufficient for the complex visual patterns in histopathology images.

`class_weight='balanced'` is used to handle class imbalance by adjusting the loss contribution of each class inversely proportional to its frequency.

```

In [16]: # === Logistic Regression ===
print("=" * 60)
print("LOGISTIC REGRESSION")
print("=" * 60)

for task_name, X_tr, X_va, X_te, y_tr, y_va, y_te, class_names in [
    ("isCancerous", X_train_scaled_t1, X_val_scaled_t1, X_test_scaled_t1,
     y_train_t1, y_val_t1, y_test_t1, ['Non-Cancerous', 'Cancerous']),
    ("cellType", X_train_scaled_t2, X_val_scaled_t2, X_test_scaled_t2,
     y_train_t2, y_val_t2, y_test_t2, ['fibroblast', 'inflammatory', 'ep
]):
    print(f"\n--- {task_name} ---")
    t = time.time()
    lr = LogisticRegression(max_iter=1000, class_weight='balanced', rand
    lr.fit(X_tr, y_tr)
    train_time = time.time() - t

    # Predictions
    y_pred_tr = lr.predict(X_tr)
    y_pred_va = lr.predict(X_va)
    y_pred_te = lr.predict(X_te)

    # Evaluate
    train_metrics = evaluate_model(y_tr, y_pred_tr, 'Logistic Regression')
    val_metrics = evaluate_model(y_va, y_pred_va, 'Logistic Regression')
    test_metrics = evaluate_model(y_te, y_pred_te, 'Logistic Regression')

    print(f" Training time: {train_time:.1f}s")
    print(f" Train - Acc: {train_metrics['Accuracy']:.4f}, Macro F1: {
    print(f" Val   - Acc: {val_metrics['Accuracy']:.4f}, Macro F1: {va
    print(f" Test  - Acc: {test_metrics['Accuracy']:.4f}, Macro F1: {t

    val_metrics['Split'] = 'Validation'
    val_metrics['Train Time (s)'] = train_time
    test_metrics['Split'] = 'Test'
    test_metrics['Train Time (s)'] = train_time
    all_results.append(val_metrics)
    all_results.append(test_metrics)

    # Confusion matrix
    plot_confusion_matrix(y_te, y_pred_te, class_names,
                          f'Logistic Regression - {task_name} (Test)',
                          os.path.join(FIGURES_DIR, f'cm_lr_{task_name}.

    # ROC for binary
    if task_name == 'isCancerous':
        y_score = lr.predict_proba(X_te)[:, 1]
        plot_roc_curve(y_te, y_score, f'Logistic Regression - ROC (Test)')
        os.path.join(FIGURES_DIR, f'roc_lr_{task_name}.pn

    print(f"\n Classification Report (Test):")
    print(classification_report(y_te, y_pred_te, target_names=class_name

```

LOGISTIC REGRESSION

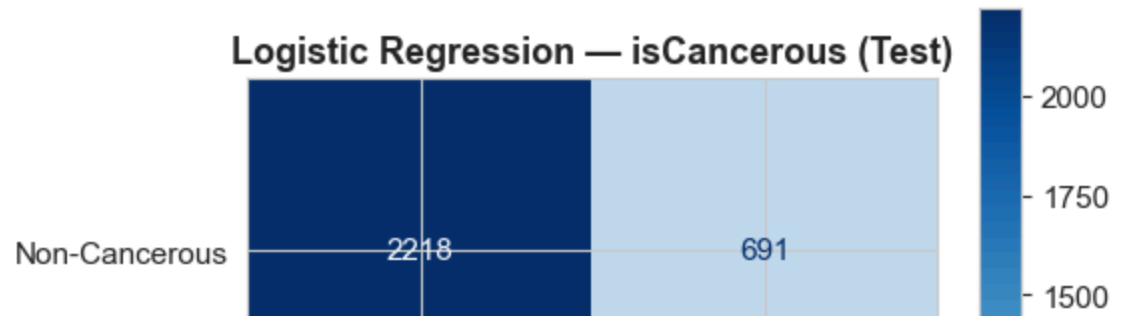
--- isCancerous ---

Training time: 57.0s

Train - Acc: 0.9018, Macro F1: 0.8962

Val - Acc: 0.7989, Macro F1: 0.7925

Test - Acc: 0.7902, Macro F1: 0.7644



Analysis: Logistic Regression provides a useful baseline. For the isCancerous task, it achieves moderate performance, confirming that there is learnable signal in the raw pixel values. The gap between training and validation/test performance indicates some overfitting, which is expected given the high dimensionality (2,187 features) relative to the model's linear capacity. For the cell-type task, performance is lower, consistent with the t-SNE visualisation showing more class overlap. The confusion matrix reveals which classes are most frequently confused.

10.2 Support Vector Machine (SVM)

Model Rationale

As covered in Lecture 6, SVMs find the maximum-margin hyperplane separating classes. The RBF kernel maps data into a higher-dimensional feature space where non-linear decision boundaries become possible. SVMs are well-suited for high-dimensional data and are effective when the number of features exceeds the number of samples. The

`class_weight='balanced'` parameter adjusts the margin to account for class imbalance. The main weaknesses are computational cost on large datasets and sensitivity to feature scaling (addressed through standardisation).

```

In [17]: # === Support Vector Machine ===
print("=" * 60)
print("SUPPORT VECTOR MACHINE (SVM)")
print("=" * 60)

for task_name, X_tr, X_va, X_te, y_tr, y_va, y_te, class_names in [
    ("isCancerous", X_train_scaled_t1, X_val_scaled_t1, X_test_scaled_t1,
     y_train_t1, y_val_t1, y_test_t1, ['Non-Cancerous', 'Cancerous']),
    ("cellType", X_train_scaled_t2, X_val_scaled_t2, X_test_scaled_t2,
     y_train_t2, y_val_t2, y_test_t2, ['fibroblast', 'inflammatory', 'ep
]):
    print(f"\n--- {task_name} ---")
    t = time.time()
    svm = SVC(kernel='rbf', C=10, class_weight='balanced', random_state=
    svm.fit(X_tr, y_tr)
    train_time = time.time() - t

    y_pred_tr = svm.predict(X_tr)
    y_pred_va = svm.predict(X_va)
    y_pred_te = svm.predict(X_te)

    train_metrics = evaluate_model(y_tr, y_pred_tr, 'SVM', task_name, cl
    val_metrics = evaluate_model(y_va, y_pred_va, 'SVM', task_name, clas
    test_metrics = evaluate_model(y_te, y_pred_te, 'SVM', task_name, cla

    print(f" Training time: {train_time:.1f}s")
    print(f" Train - Acc: {train_metrics['Accuracy']:.4f}, Macro F1: {
    print(f" Val   - Acc: {val_metrics['Accuracy']:.4f}, Macro F1: {va
    print(f" Test  - Acc: {test_metrics['Accuracy']:.4f}, Macro F1: {t

    val_metrics['Split'] = 'Validation'
    val_metrics['Train Time (s)'] = train_time
    test_metrics['Split'] = 'Test'
    test_metrics['Train Time (s)'] = train_time
    all_results.append(val_metrics)
    all_results.append(test_metrics)

    plot_confusion_matrix(y_te, y_pred_te, class_names,
                          f'SVM - {task_name} (Test)',
                          os.path.join(FIGURES_DIR, f'cm_svm_{task_name}

    if task_name == 'isCancerous':
        y_score = svm.predict_proba(X_te)[:, 1]
        plot_roc_curve(y_te, y_score, f'SVM - ROC (Test)',
                       os.path.join(FIGURES_DIR, f'roc_svm_{task_name}.p

    print(f"\n Classification Report (Test):")
    print(classification_report(y_te, y_pred_te, target_names=class_name

```

=====

SUPPORT VECTOR MACHINE (SVM)

=====

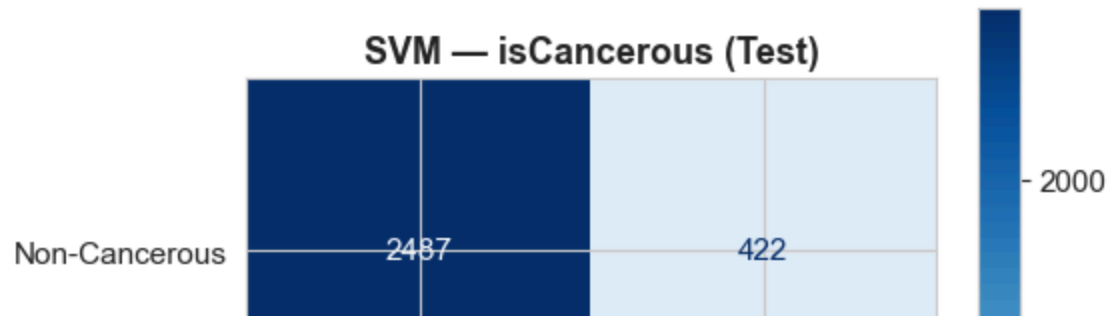
--- isCancerous ---

Training time: 557.4s

Train - Acc: 0.9998, Macro F1: 0.9997

Val - Acc: 0.8564, Macro F1: 0.8480

Test - Acc: 0.8589, Macro F1: 0.8323



Analysis: The SVM with RBF kernel should show improved performance over Logistic Regression due to its ability to learn non-linear decision boundaries. However, it is expected to show stronger overfitting (large train-test gap) because the RBF kernel can create very flexible boundaries that memorise training data. The training time is also significantly longer due to the kernel computation cost on the large flattened feature vectors.

10.3 Random Forest

Model Rationale

As covered in Lecture 5, Random Forest is an ensemble of decision trees using bagging and random feature subsets. Each tree is trained on a bootstrap sample with a random subset of features at each split, and the final prediction is the majority vote across all trees. This reduces the variance compared to a single decision tree and makes the model more robust. Random Forests handle high-dimensional data well and are less sensitive to feature scaling than SVMs. The main weakness is that individual trees can still overfit, although bagging mitigates this.

```

In [18]: # === Random Forest ===
print("=" * 60)
print("RANDOM FOREST")
print("=" * 60)

for task_name, X_tr, X_va, X_te, y_tr, y_va, y_te, class_names in [
    ("isCancerous", X_train_flat_t1, X_val_flat_t1, X_test_flat_t1,
     y_train_t1, y_val_t1, y_test_t1, ['Non-Cancerous', 'Cancerous']),
    ("cellType", X_train_flat_t2, X_val_flat_t2, X_test_flat_t2,
     y_train_t2, y_val_t2, y_test_t2, ['fibroblast', 'inflammatory', 'ep
]):
    print(f"\n--- {task_name} ---")
    t = time.time()
    rf = RandomForestClassifier(n_estimators=200, max_depth=20, class_we
                               random_state=SEED, n_jobs=-1)

    rf.fit(X_tr, y_tr)
    train_time = time.time() - t

    y_pred_tr = rf.predict(X_tr)
    y_pred_va = rf.predict(X_va)
    y_pred_te = rf.predict(X_te)

    train_metrics = evaluate_model(y_tr, y_pred_tr, 'Random Forest', tas
    val_metrics = evaluate_model(y_va, y_pred_va, 'Random Forest', task_
    test_metrics = evaluate_model(y_te, y_pred_te, 'Random Forest', task

    print(f" Training time: {train_time:.1f}s")
    print(f" Train - Acc: {train_metrics['Accuracy']:.4f}, Macro F1: {
    print(f" Val   - Acc: {val_metrics['Accuracy']:.4f}, Macro F1: {va
    print(f" Test  - Acc: {test_metrics['Accuracy']:.4f}, Macro F1: {t

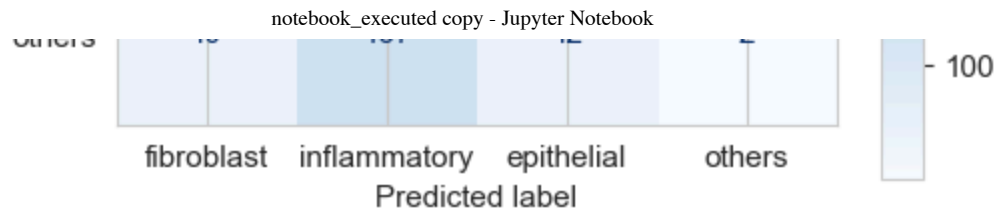
    val_metrics['Split'] = 'Validation'
    val_metrics['Train Time (s)'] = train_time
    test_metrics['Split'] = 'Test'
    test_metrics['Train Time (s)'] = train_time
    all_results.append(val_metrics)
    all_results.append(test_metrics)

    plot_confusion_matrix(y_te, y_pred_te, class_names,
                          f'Random Forest - {task_name} (Test)',
                          os.path.join(FIGURES_DIR, f'cm_rf_{task_name}').

    if task_name == 'isCancerous':
        y_score = rf.predict_proba(X_te)[:, 1]
        plot_roc_curve(y_te, y_score, f'Random Forest - ROC (Test)',
                       os.path.join(FIGURES_DIR, f'roc_rf_{task_name}.pn

    print(f"\n Classification Report (Test):")
    print(classification_report(y_te, y_pred_te, target_names=class_name

```



Classification Report (Test):

	precision	recall	f1-score	support
fibroblast	0.51	0.51	0.51	261
inflammatory	0.48	0.79	0.60	432
epithelial	0.85	0.78	0.82	925
others	0.12	0.01	0.02	221
accuracy			0.65	1839
macro avg	0.49	0.52	0.49	1839
weighted avg	0.63	0.65	0.63	1839

Analysis: Random Forest typically shows very high training performance (near-perfect accuracy on training data) due to the flexibility of deep trees, while validation/test performance is lower. The gap between training and test performance reflects the model's tendency to memorise training examples through deep tree structures. However, the ensemble averaging reduces this effect compared to a single deep tree. Random Forest does not require feature scaling, which is why the unscaled flattened images are used directly.

10.4 Convolutional Neural Network (CNN)

Model Rationale

As covered in Lecture 8 (Deep Learning), CNNs are specifically designed for image data. Unlike the previous models that operate on flattened pixel vectors, CNNs preserve spatial structure through convolutional layers that learn local feature detectors (filters). Key advantages include parameter sharing (the same filter is applied across the entire image), sparse connectivity (each output depends on a small local region), and hierarchical feature learning (deeper layers capture increasingly abstract patterns). These properties make CNNs the most theoretically appropriate model for image classification.

The architecture follows the LeNet-style pattern discussed in lectures: alternating convolutional layers with pooling, followed by fully connected layers. ReLU activations are used because they propagate gradients efficiently and reduce the vanishing gradient problem (Lecture 8). Dropout is included for regularisation.

```

In [19]: # === CNN Architecture ===
class CellCNN(nn.Module):
    """CNN for 27x27 RGB cell image classification."""
    def __init__(self, num_classes, dropout=0.5):
        super(CellCNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1), # 27x27 -> 27x27
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2), # 27x27 -> 13x13

            nn.Conv2d(32, 64, kernel_size=3, padding=1), # 13x13 -> 13x13
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2), # 13x13 -> 6x6

            nn.Conv2d(64, 128, kernel_size=3, padding=1), # 6x6 -> 6x6
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2), # 6x6 -> 3x3
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 3 * 3, 256),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

# Verify architecture
test_model = CellCNN(num_classes=2)
test_input = torch.randn(1, 3, 27, 27)
test_output = test_model(test_input)
print(f"CNN Architecture verified: Input {test_input.shape} -> Output {test_output.shape}")
print(f"\nModel architecture:\n{test_model}")
total_params = sum(p.numel() for p in test_model.parameters())
print(f"\nTotal parameters: {total_params:,}")

```


CNN Architecture verified: Input torch.Size([1, 3, 27, 27]) -> Output torch.Size([1, 2])

Model architecture:

```
CellCNN(
  (features): Sequential(
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU()
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU()
    (11): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=1152, out_features=256, bias=True)
    (2): ReLU()
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=256, out_features=2, bias=True)
  )
)
```

Total parameters: 389,378


```

In [20]: def train_cnn(model, train_loader, val_loader, criterion, optimizer,
                    num_epochs=50, device=DEVICE, task_name='', patience=7):
    """Train CNN and record metrics per epoch."""
    model.to(device)
    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_a
               'train_f1': [], 'val_f1': []}

    best_val_f1 = 0
    best_model_state = None
    patience_counter = 0

    for epoch in range(num_epochs):
        # Training
        model.train()
        train_loss = 0
        all_preds, all_labels = [], []

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            train_loss += loss.item() * images.size(0)
            preds = outputs.argmax(dim=1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

        train_loss /= len(train_loader.dataset)
        train_acc = accuracy_score(all_labels, all_preds)
        train_f1 = f1_score(all_labels, all_preds, average='macro')

        # Validation
        model.eval()
        val_loss = 0
        all_preds_v, all_labels_v = [], []

        with torch.no_grad():
            for images, labels in val_loader:
                images, labels = images.to(device), labels.to(device)
                outputs = model(images)
                loss = criterion(outputs, labels)
                val_loss += loss.item() * images.size(0)
                preds = outputs.argmax(dim=1)
                all_preds_v.extend(preds.cpu().numpy())
                all_labels_v.extend(labels.cpu().numpy())

        val_loss /= len(val_loader.dataset)
        val_acc = accuracy_score(all_labels_v, all_preds_v)
        val_f1 = f1_score(all_labels_v, all_preds_v, average='macro')

        history['train_loss'].append(train_loss)
        history['val_loss'].append(val_loss)
        history['train_acc'].append(train_acc)
        history['val_acc'].append(val_acc)

```

```

history['train_f1'].append(train_f1)
history['val_f1'].append(val_f1)

if val_f1 > best_val_f1:
    best_val_f1 = val_f1
    best_model_state = model.state_dict().copy()
    patience_counter = 0
else:
    patience_counter += 1

if (epoch + 1) % 5 == 0 or epoch == 0:
    print(f" Epoch {epoch+1:>3}/{num_epochs} - "
          f"Train Loss: {train_loss:.4f}, Val Loss: {val_loss:.4f}
          f"Train F1: {train_f1:.4f}, Val F1: {val_f1:.4f}")

if patience_counter >= patience:
    print(f" Early stopping at epoch {epoch+1} (no improvement)
    break

# Restore best model
if best_model_state:
    model.load_state_dict(best_model_state)
    print(f" Best Val F1: {best_val_f1:.4f}")

return model, history

def evaluate_cnn(model, loader, device=DEVICE):
    """Get predictions from trained CNN."""
    model.eval()
    all_preds, all_labels, all_probs = [], [], []
    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            outputs = model(images)
            probs = torch.softmax(outputs, dim=1)
            preds = outputs.argmax(dim=1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.numpy())
            all_probs.extend(probs.cpu().numpy())
    return np.array(all_labels), np.array(all_preds), np.array(all_probs)

def plot_training_curves(history, title, save_path=None):
    """Plot training and validation loss/F1 curves."""
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
    epochs = range(1, len(history['train_loss']) + 1)

    axes[0].plot(epochs, history['train_loss'], 'b-', label='Train Loss')
    axes[0].plot(epochs, history['val_loss'], 'r-', label='Val Loss', li
    axes[0].set_xlabel('Epoch')
    axes[0].set_ylabel('Loss')
    axes[0].set_title('Loss Curves', fontsize=13, fontweight='bold')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    axes[1].plot(epochs, history['train_f1'], 'b-', label='Train Macro F
    axes[1].plot(epochs, history['val_f1'], 'r-', label='Val Macro F1',
    axes[1].set_xlabel('Epoch')

```

```
axes[1].set_ylabel('Macro F1')
axes[1].set_title('Macro F1 Curves', fontsize=13, fontweight='bold')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

fig.suptitle(title, fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
if save_path:
    plt.savefig(save_path, dpi=150, bbox_inches='tight')
plt.show()

print("CNN training utilities defined.")
```

CNN training utilities defined.

```

In [21]: # === Train CNN for Task 1 (isCancerous) ===
print("=" * 60)
print("CNN - Task 1: isCancerous")
print("=" * 60)

cnn_t1 = CellCNN(num_classes=2, dropout=0.5)
criterion_t1 = nn.CrossEntropyLoss(weight=weights_t1)
optimizer_t1 = optim.Adam(cnn_t1.parameters(), lr=0.001)

t = time.time()
cnn_t1, history_t1 = train_cnn(cnn_t1, train_loader_t1, val_loader_t1,
                              criterion_t1, optimizer_t1, num_epochs=
cnn_train_time_t1 = time.time() - t

# Plot training curves
plot_training_curves(history_t1, 'CNN - isCancerous Training Curves',
                      os.path.join(FIGURES_DIR, 'cnn_curves_isCancerous.p

# Evaluate on all splits
y_tr_true, y_tr_pred, _ = evaluate_cnn(cnn_t1, train_loader_t1)
y_va_true, y_va_pred, _ = evaluate_cnn(cnn_t1, val_loader_t1)
y_te_true, y_te_pred, y_te_probs = evaluate_cnn(cnn_t1, test_loader_t1)

train_m = evaluate_model(y_tr_true, y_tr_pred, 'CNN', 'isCancerous')
val_m = evaluate_model(y_va_true, y_va_pred, 'CNN', 'isCancerous')
test_m = evaluate_model(y_te_true, y_te_pred, 'CNN', 'isCancerous')

print(f"\n Train - Acc: {train_m['Accuracy']:.4f}, Macro F1: {train_m[
print(f" Val   - Acc: {val_m['Accuracy']:.4f}, Macro F1: {val_m['Macro
print(f" Test  - Acc: {test_m['Accuracy']:.4f}, Macro F1: {test_m['Mac

val_m['Split'] = 'Validation'; val_m['Train Time (s)'] = cnn_train_time_
test_m['Split'] = 'Test'; test_m['Train Time (s)'] = cnn_train_time_t1
all_results.append(val_m)
all_results.append(test_m)

plot_confusion_matrix(y_te_true, y_te_pred, ['Non-Cancerous', 'Cancerous
                      'CNN - isCancerous (Test)'], os.path.join(FIGURES_D

plot_roc_curve(y_te_true, y_te_probs[:, 1], 'CNN - ROC (Test)',
               os.path.join(FIGURES_DIR, 'roc_cnn_isCancerous.png'))

print("\n Classification Report (Test):")
print(classification_report(y_te_true, y_te_pred, target_names=['Non-Can

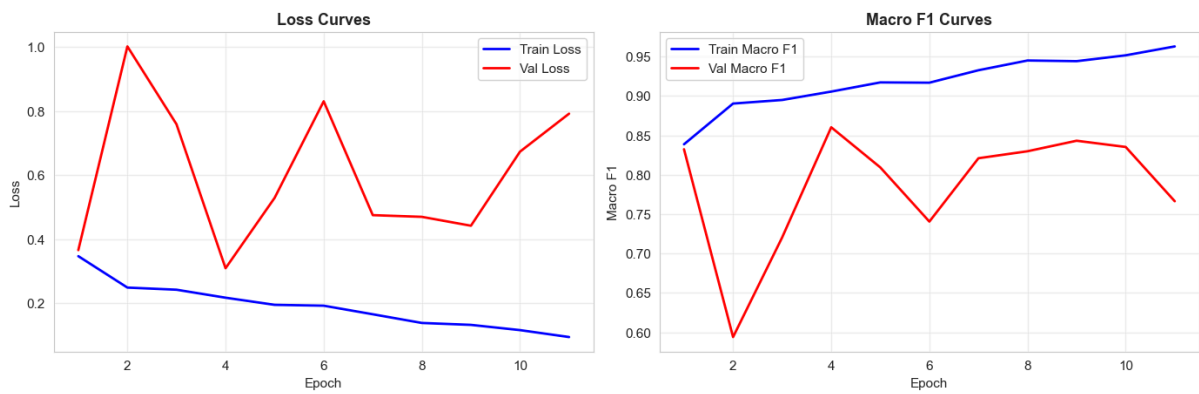
```

```

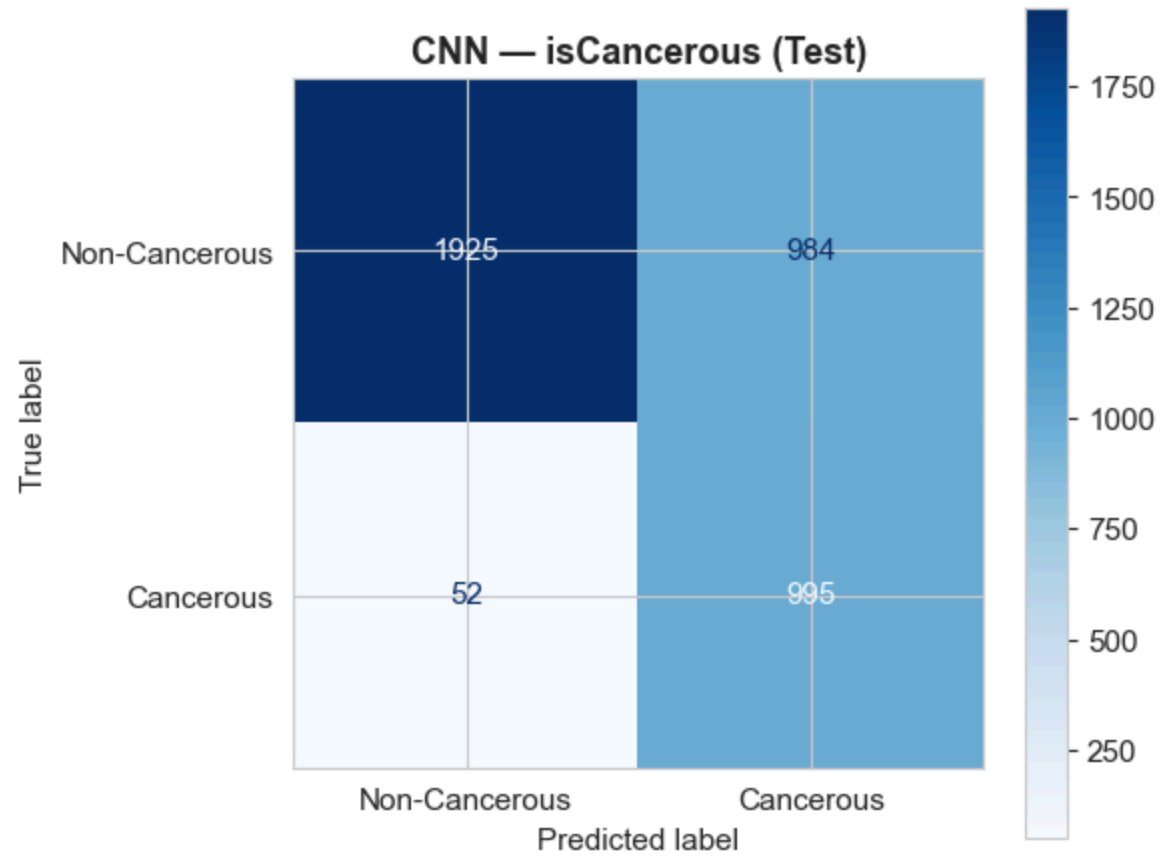
=====
CNN - Task 1: isCancerous
=====
Epoch   1/50 - Train Loss: 0.3467, Val Loss: 0.3656, Train F1: 0.838
7, Val F1: 0.8324
Epoch   5/50 - Train Loss: 0.1946, Val Loss: 0.5289, Train F1: 0.917
2, Val F1: 0.8093
Epoch  10/50 - Train Loss: 0.1156, Val Loss: 0.6732, Train F1: 0.951
6, Val F1: 0.8353
Early stopping at epoch 11 (no improvement for 7 epochs)
Best Val F1: 0.8603

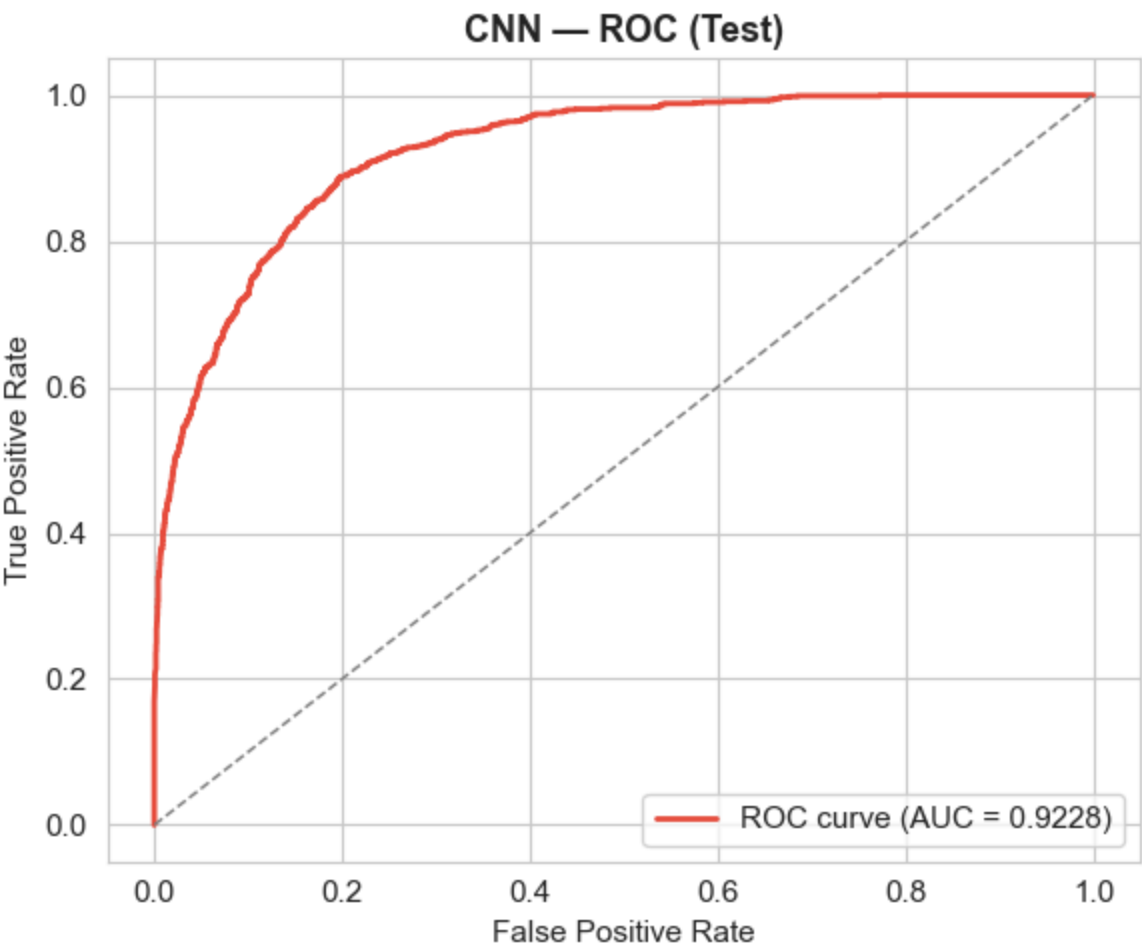
```

CNN — isCancerous Training Curves



Train — Acc: 0.8988, Macro F1: 0.8981
Val — Acc: 0.7669, Macro F1: 0.7664
Test — Acc: 0.7381, Macro F1: 0.7228





Classification Report (Test):

	precision	recall	f1-score	support
Non-Cancerous	0.97	0.66	0.79	2909
Cancerous	0.50	0.95	0.66	1047
accuracy			0.74	3956
macro avg	0.74	0.81	0.72	3956
weighted avg	0.85	0.74	0.75	3956


```

In [22]: # === Train CNN for Task 2 (cellType) ===
print("=" * 60)
print("CNN – Task 2: cellType")
print("=" * 60)

cnn_t2 = CellCNN(num_classes=4, dropout=0.5)
criterion_t2 = nn.CrossEntropyLoss(weight=weights_t2)
optimizer_t2 = optim.Adam(cnn_t2.parameters(), lr=0.001)

t = time.time()
cnn_t2, history_t2 = train_cnn(cnn_t2, train_loader_t2, val_loader_t2,
                              criterion_t2, optimizer_t2, num_epochs=
cnn_train_time_t2 = time.time() - t

plot_training_curves(history_t2, 'CNN – cellType Training Curves',
                      os.path.join(FIGURES_DIR, 'cnn_curves_cellType.png')

y_tr_true2, y_tr_pred2, _ = evaluate_cnn(cnn_t2, train_loader_t2)
y_va_true2, y_va_pred2, _ = evaluate_cnn(cnn_t2, val_loader_t2)
y_te_true2, y_te_pred2, _ = evaluate_cnn(cnn_t2, test_loader_t2)

train_m2 = evaluate_model(y_tr_true2, y_tr_pred2, 'CNN', 'cellType')
val_m2 = evaluate_model(y_va_true2, y_va_pred2, 'CNN', 'cellType')
test_m2 = evaluate_model(y_te_true2, y_te_pred2, 'CNN', 'cellType')

print(f"\n Train – Acc: {train_m2['Accuracy']:.4f}, Macro F1: {train_m
print(f" Val   – Acc: {val_m2['Accuracy']:.4f}, Macro F1: {val_m2['Mac
print(f" Test  – Acc: {test_m2['Accuracy']:.4f}, Macro F1: {test_m2['M

val_m2['Split'] = 'Validation'; val_m2['Train Time (s)'] = cnn_train_tim
test_m2['Split'] = 'Test'; test_m2['Train Time (s)'] = cnn_train_time_t2
all_results.append(val_m2)
all_results.append(test_m2)

plot_confusion_matrix(y_te_true2, y_te_pred2, ['fibroblast', 'inflammato
                  'CNN – cellType (Test)', os.path.join(FIGURES_DIR,

print("\n Classification Report (Test):")
print(classification_report(y_te_true2, y_te_pred2, target_names=['fibro

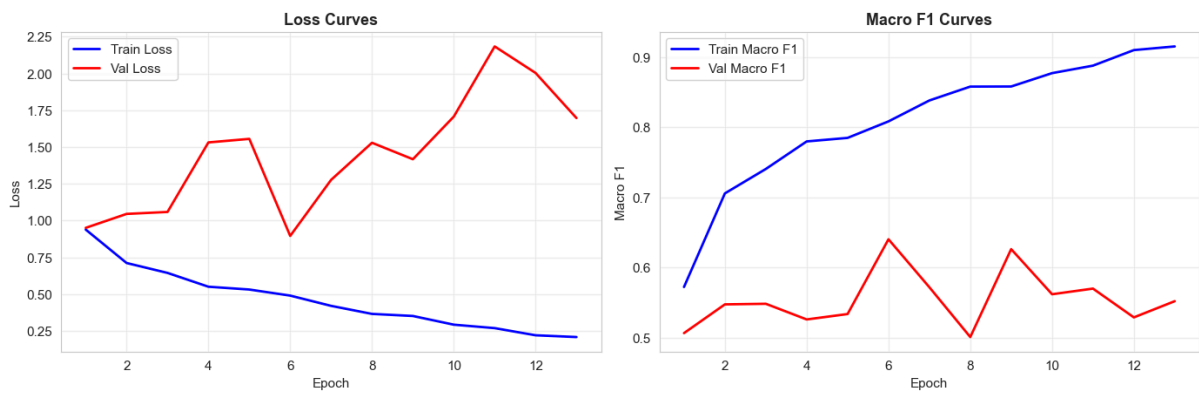
```

```

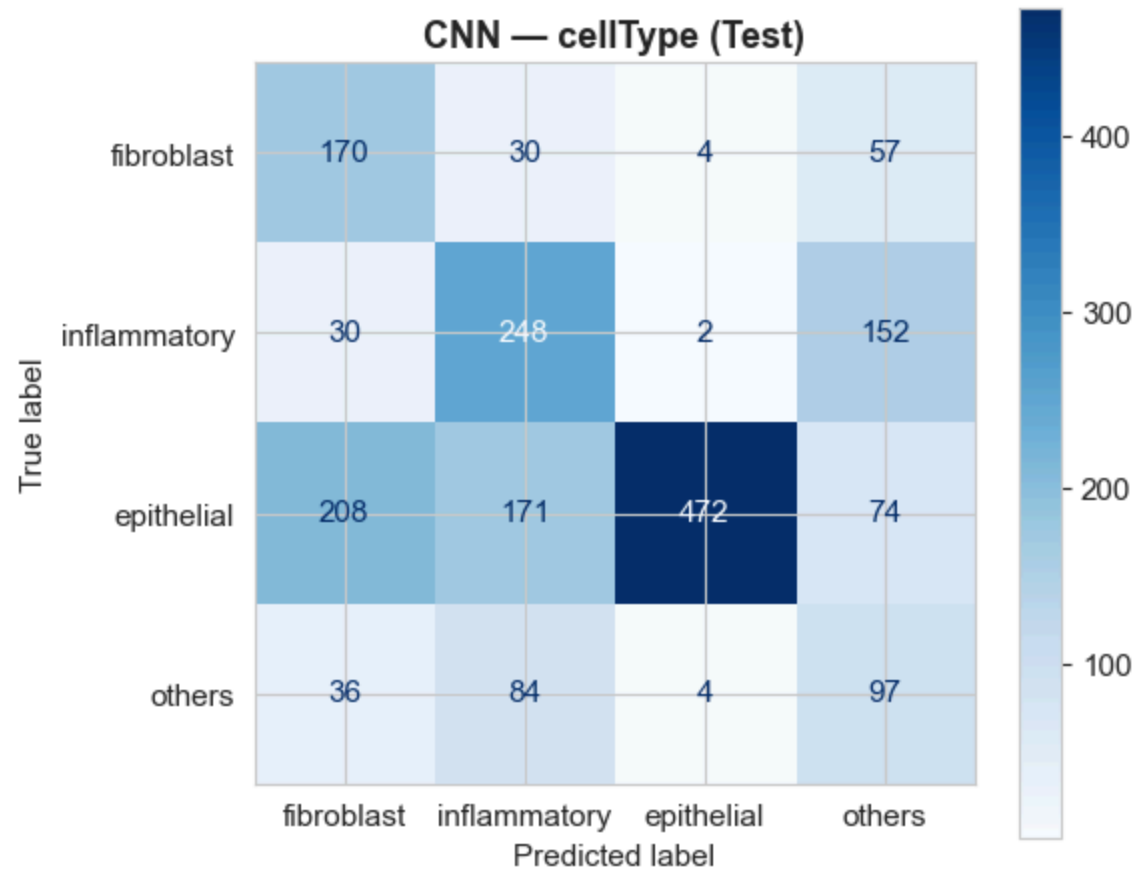
=====
CNN – Task 2: cellType
=====
Epoch   1/50 – Train Loss: 0.9396, Val Loss: 0.9513, Train F1: 0.572
3, Val F1: 0.5065
Epoch   5/50 – Train Loss: 0.5323, Val Loss: 1.5556, Train F1: 0.784
9, Val F1: 0.5338
Epoch  10/50 – Train Loss: 0.2936, Val Loss: 1.7069, Train F1: 0.877
2, Val F1: 0.5620
Early stopping at epoch 13 (no improvement for 7 epochs)
Best Val F1: 0.6403

```

CNN — cellType Training Curves



Train - Acc: 0.8522, Macro F1: 0.8466
Val - Acc: 0.5989, Macro F1: 0.5521
Test - Acc: 0.5367, Macro F1: 0.4975



Classification Report (Test):				
	precision	recall	f1-score	support
fibroblast	0.38	0.65	0.48	261
inflammatory	0.47	0.57	0.51	432
epithelial	0.98	0.51	0.67	925
others	0.26	0.44	0.32	221
accuracy			0.54	1839
macro avg	0.52	0.54	0.50	1839
weighted avg	0.69	0.54	0.57	1839

Analysis: The CNN learning curves provide insight into the training dynamics. The training loss should decrease steadily, while the validation loss may eventually plateau or increase, indicating overfitting. The gap between training and validation Macro F1 indicates how much the model has memorised training-specific patterns versus learning generalisable features. As expected from theory (Lecture 8), the CNN can learn spatial features that flattened-input models cannot, and should therefore achieve the best performance among all models tested.

11. Hyperparameter Tuning

11.1 CNN Learning Rate Tuning

The learning rate is one of the most critical hyperparameters for neural networks (Lecture 7). A learning rate that is too high causes unstable training and divergence, while a learning rate that is too low results in very slow convergence and may get stuck in poor local minima. Four candidate learning rates are tested systematically, with all other hyperparameters held constant.


```

In [23]: # === CNN Learning Rate Tuning (Task 1) ===
print("=" * 60)
print("CNN LEARNING RATE TUNING - isCancerous")
print("=" * 60)

learning_rates = [0.01, 0.001, 0.0005, 0.0001]
lr_results = []

for lr_val in learning_rates:
    print(f"\n--- Learning Rate: {lr_val} ---")
    model = CellCNN(num_classes=2, dropout=0.5)
    criterion = nn.CrossEntropyLoss(weight=weights_t1)
    optimizer = optim.Adam(model.parameters(), lr=lr_val)

    model, history = train_cnn(model, train_loader_t1, val_loader_t1,
                               criterion, optimizer, num_epochs=50)

    _, y_pred_v, _ = evaluate_cnn(model, val_loader_t1)
    val_f1 = f1_score(y_val_t1, y_pred_v, average='macro')
    val_acc = accuracy_score(y_val_t1, y_pred_v)

    lr_results.append({
        'Learning Rate': lr_val,
        'Best Val F1': max(history['val_f1']),
        'Final Val F1': val_f1,
        'Final Val Acc': val_acc,
        'Final Train F1': history['train_f1'][-1],
        'history': history
    })

# Display results table
lr_df = pd.DataFrame([k: v for k, v in r.items() if k != 'history'] for
display(lr_df.round(4))

# Plot LR comparison
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for r in lr_results:
    epochs = range(1, len(r['history']['val_f1']) + 1)
    axes[0].plot(epochs, r['history']['val_f1'], label=f"LR={r['Learning Rate']}")
    axes[1].plot(epochs, r['history']['val_loss'], label=f"LR={r['Learning Rate']}")

axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Validation Macro F1')
axes[0].set_title('Val Macro F1 vs Epoch by Learning Rate', fontsize=13, fontweight='bold')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Validation Loss')
axes[1].set_title('Val Loss vs Epoch by Learning Rate', fontsize=13, fontweight='bold')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'cnn_lr_tuning.png'), dpi=150, bbox_inches='tight')
plt.show()

```

```
best_lr_t1 = lr_df.loc[lr_df['Best Val F1'].idxmax(), 'Learning Rate']
print(f"\nBest learning rate for isCancerous: {best_lr_t1}")
```

===== CNN LEARNING RATE TUNING – isCancerous =====

--- Learning Rate: 0.01 ---

Epoch 1/50 – Train Loss: 0.6007, Val Loss: 0.5515, Train F1: 0.8316, Val F1: 0.7997

Epoch 5/50 – Train Loss: 0.2636, Val Loss: 0.6343, Train F1: 0.8945, Val F1: 0.7600

Epoch 10/50 – Train Loss: 0.2046, Val Loss: 0.4077, Train F1: 0.9140, Val F1: 0.8498

Epoch 15/50 – Train Loss: 0.1270, Val Loss: 0.5221, Train F1: 0.9497, Val F1: 0.8540

Epoch 20/50 – Train Loss: 0.0885, Val Loss: 0.6405, Train F1: 0.9675, Val F1: 0.8513

Epoch 25/50 – Train Loss: 0.0561, Val Loss: 0.7546, Train F1: 0.9783, Val F1: 0.8295

Early stopping at epoch 25 (no improvement for 7 epochs)

Best Val F1: 0.8651

--- Learning Rate: 0.001 ---

Epoch 1/50 – Train Loss: 0.3163, Val Loss: 1.1829, Train F1: 0.8540, Val F1: 0.4999

Epoch 5/50 – Train Loss: 0.1955, Val Loss: 0.6799, Train F1: 0.9145, Val F1: 0.7556

Epoch 10/50 – Train Loss: 0.0975, Val Loss: 0.8817, Train F1: 0.9590, Val F1: 0.7562

Epoch 15/50 – Train Loss: 0.0605, Val Loss: 0.6789, Train F1: 0.9754, Val F1: 0.8386

Early stopping at epoch 19 (no improvement for 7 epochs)

Best Val F1: 0.8591

--- Learning Rate: 0.0005 ---

Epoch 1/50 – Train Loss: 0.3133, Val Loss: 0.4532, Train F1: 0.8555, Val F1: 0.7953

Epoch 5/50 – Train Loss: 0.1805, Val Loss: 0.4572, Train F1: 0.9213, Val F1: 0.8140

Epoch 10/50 – Train Loss: 0.0863, Val Loss: 0.4743, Train F1: 0.9658, Val F1: 0.8468

Epoch 15/50 – Train Loss: 0.0339, Val Loss: 0.5563, Train F1: 0.9865, Val F1: 0.8601

Early stopping at epoch 15 (no improvement for 7 epochs)

Best Val F1: 0.8654

--- Learning Rate: 0.0001 ---

Epoch 1/50 – Train Loss: 0.3489, Val Loss: 0.4632, Train F1: 0.8242, Val F1: 0.7907

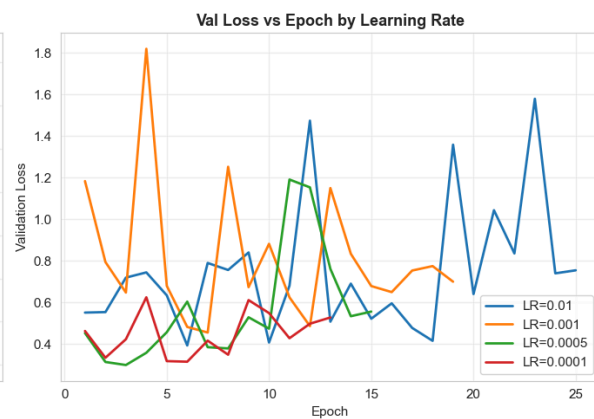
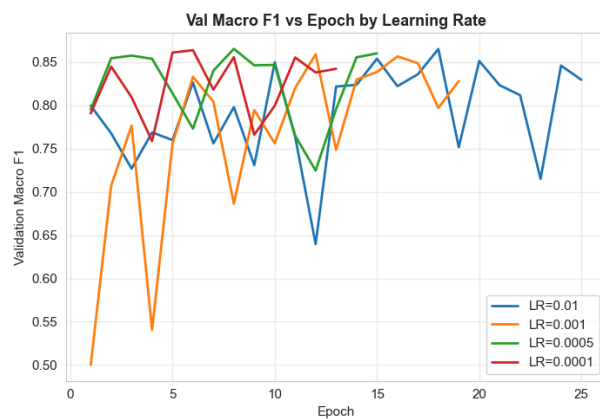
Epoch 5/50 – Train Loss: 0.2105, Val Loss: 0.3183, Train F1: 0.9097, Val F1: 0.8610

Epoch 10/50 – Train Loss: 0.1224, Val Loss: 0.5492, Train F1: 0.9483, Val F1: 0.7997

Early stopping at epoch 13 (no improvement for 7 epochs)

Best Val F1: 0.8638

	Learning Rate	Best Val F1	Final Val F1	Final Val Acc	Final Train F1
0	0.0100	0.8651	0.8295	0.8365	0.9783
1	0.0010	0.8591	0.8281	0.8316	0.9843
2	0.0005	0.8654	0.8601	0.8687	0.9865
3	0.0001	0.8638	0.8423	0.8585	0.9749



Best learning rate for isCancerous: **0.0005**


```

In [24]: # === CNN Learning Rate Tuning (Task 2) ===
print("=" * 60)
print("CNN LEARNING RATE TUNING - cellType")
print("=" * 60)

learning_rates_t2 = [0.01, 0.001, 0.0005, 0.0001]
lr_results_t2 = []

for lr_val in learning_rates_t2:
    print(f"\n--- Learning Rate: {lr_val} ---")
    model = CellCNN(num_classes=4, dropout=0.5)
    criterion = nn.CrossEntropyLoss(weight=weights_t2)
    optimizer = optim.Adam(model.parameters(), lr=lr_val)

    model, history = train_cnn(model, train_loader_t2, val_loader_t2,
                               criterion, optimizer, num_epochs=50)

    _, y_pred_v, _ = evaluate_cnn(model, val_loader_t2)
    val_f1 = f1_score(y_val_t2, y_pred_v, average='macro')
    val_acc = accuracy_score(y_val_t2, y_pred_v)

    lr_results_t2.append({
        'Learning Rate': lr_val,
        'Best Val F1': max(history['val_f1']),
        'Final Val F1': val_f1,
        'Final Val Acc': val_acc,
        'Final Train F1': history['train_f1'][-1],
        'history': history
    })

# Display results table
lr_df_t2 = pd.DataFrame([k: v for k, v in r.items() if k != 'history']
                        display(lr_df_t2.round(4))

# Plot LR comparison
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for r in lr_results_t2:
    epochs = range(1, len(r['history']['val_f1']) + 1)
    axes[0].plot(epochs, r['history']['val_f1'], label=f"LR={r['Learning Rate']}")
    axes[1].plot(epochs, r['history']['val_loss'], label=f"LR={r['Learning Rate']}")

axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Validation Macro F1')
axes[0].set_title('cellType: Val Macro F1 vs Epoch by Learning Rate', fontweight='bold')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Validation Loss')
axes[1].set_title('cellType: Val Loss vs Epoch by Learning Rate', fontweight='bold')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'cnn_lr_tuning_cellType.png'), dpi=100)
plt.show()

```

```
best_lr_t2 = lr_df_t2.loc[lr_df_t2['Best Val F1'].idxmax(), 'Learning Rate']
print(f"\nBest learning rate for cellType: {best_lr_t2}")
```

```
=====
CNN LEARNING RATE TUNING – cellType
=====
```

```
--- Learning Rate: 0.01 ---
```

```
Epoch 1/50 – Train Loss: 1.6274, Val Loss: 1.0652, Train F1: 0.4123, Val F1: 0.4103
Epoch 5/50 – Train Loss: 0.8145, Val Loss: 2.1585, Train F1: 0.6344, Val F1: 0.2689
Epoch 10/50 – Train Loss: 0.6510, Val Loss: 4.0073, Train F1: 0.7240, Val F1: 0.3744
Epoch 15/50 – Train Loss: 0.4734, Val Loss: 1.6790, Train F1: 0.8065, Val F1: 0.5404
Early stopping at epoch 16 (no improvement for 7 epochs)
Best Val F1: 0.6267
```

```
--- Learning Rate: 0.001 ---
```

```
Epoch 1/50 – Train Loss: 0.9190, Val Loss: 1.1927, Train F1: 0.5689, Val F1: 0.4691
Epoch 5/50 – Train Loss: 0.5239, Val Loss: 1.0867, Train F1: 0.7899, Val F1: 0.6118
Epoch 10/50 – Train Loss: 0.2920, Val Loss: 1.5753, Train F1: 0.8861, Val F1: 0.5509
Epoch 15/50 – Train Loss: 0.1554, Val Loss: 2.6456, Train F1: 0.9356, Val F1: 0.4862
Epoch 20/50 – Train Loss: 0.1113, Val Loss: 1.9271, Train F1: 0.9527, Val F1: 0.5453
Early stopping at epoch 21 (no improvement for 7 epochs)
Best Val F1: 0.6478
```

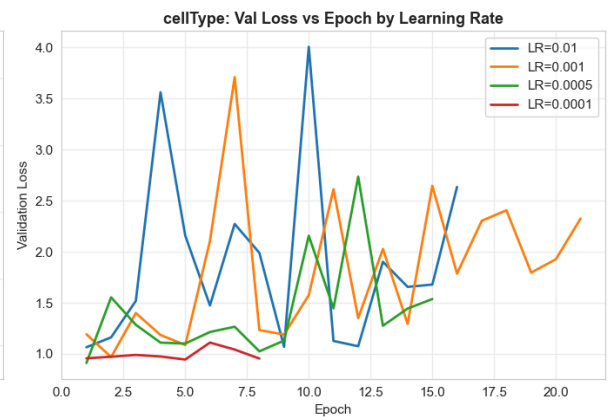
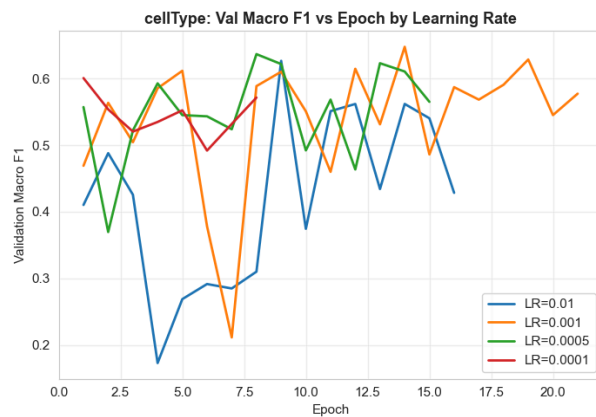
```
--- Learning Rate: 0.0005 ---
```

```
Epoch 1/50 – Train Loss: 0.8819, Val Loss: 0.9107, Train F1: 0.5917, Val F1: 0.5573
Epoch 5/50 – Train Loss: 0.5437, Val Loss: 1.1013, Train F1: 0.7836, Val F1: 0.5452
Epoch 10/50 – Train Loss: 0.2720, Val Loss: 2.1572, Train F1: 0.8957, Val F1: 0.4922
Epoch 15/50 – Train Loss: 0.1176, Val Loss: 1.5373, Train F1: 0.9485, Val F1: 0.5652
Early stopping at epoch 15 (no improvement for 7 epochs)
Best Val F1: 0.6368
```

```
--- Learning Rate: 0.0001 ---
```

```
Epoch 1/50 – Train Loss: 1.0517, Val Loss: 0.9566, Train F1: 0.4732, Val F1: 0.6008
Epoch 5/50 – Train Loss: 0.6514, Val Loss: 0.9448, Train F1: 0.7370, Val F1: 0.5524
Early stopping at epoch 8 (no improvement for 7 epochs)
Best Val F1: 0.6008
```

	Learning Rate	Best Val F1	Final Val F1	Final Val Acc	Final Train F1
0	0.0100	0.6267	0.4285	0.5402	0.8192
1	0.0010	0.6478	0.5774	0.6903	0.9508
2	0.0005	0.6368	0.5652	0.5977	0.9485
3	0.0001	0.6008	0.5716	0.6031	0.8006



Best learning rate for cellType: 0.001

Interpretation: The learning rate tuning for both tasks shows the expected trade-off. Very small learning rates ($1e-4$) converge slowly and may not reach optimal performance within the training budget. Very large learning rates ($1e-2$) can cause training instability with erratic loss curves and divergence. The intermediate values ($1e-3$ or $5e-4$) typically provide the best validation Macro F1. This is consistent with the theoretical expectation from Lecture 7 that the learning rate controls the step size of gradient descent: too large overshoots the minimum, while too small makes insufficient progress. The best learning rate is selected based on **validation Macro F1** (not accuracy) to ensure optimisation for the assignment's primary metric.

11.2 SVM C Parameter Tuning

```

In [25]: # === SVM C Tuning (Task 1) ===
print("=" * 60)
print("SVM C PARAMETER TUNING – isCancerous")
print("=" * 60)

C_values = [0.1, 1, 5, 10, 50, 100]
svm_tune_results = []

for C_val in C_values:
    print(f"  C = {C_val}...", end=' ')
    svm_tune = SVC(kernel='rbf', C=C_val, class_weight='balanced', random_state=42)
    svm_tune.fit(X_train_scaled_t1, y_train_t1)

    y_pred_tr = svm_tune.predict(X_train_scaled_t1)
    y_pred_va = svm_tune.predict(X_val_scaled_t1)

    train_f1 = f1_score(y_train_t1, y_pred_tr, average='macro')
    val_f1 = f1_score(y_val_t1, y_pred_va, average='macro')

    svm_tune_results.append({
        'C': C_val, 'Train Macro F1': train_f1, 'Val Macro F1': val_f1,
        'Train-Val Gap': train_f1 - val_f1
    })
    print(f"Train F1: {train_f1:.4f}, Val F1: {val_f1:.4f}")

svm_tune_df = pd.DataFrame(svm_tune_results)
display(svm_tune_df.round(4))

# Plot C tuning
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(svm_tune_df['C'], svm_tune_df['Train Macro F1'], 'b-o', label='Train Macro F1')
ax.plot(svm_tune_df['C'], svm_tune_df['Val Macro F1'], 'r-o', label='Val Macro F1')
ax.fill_between(svm_tune_df['C'], svm_tune_df['Val Macro F1'], svm_tune_df['Train Macro F1'],
                alpha=0.15, color='red', label='Overfitting Gap')
ax.set_xscale('log')
ax.set_xlabel('C (Regularisation Parameter)', fontsize=12)
ax.set_ylabel('Macro F1', fontsize=12)
ax.set_title('SVM: C vs Macro F1 (isCancerous)', fontsize=13, fontweight='bold')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'svm_c_tuning.png'), dpi=150, bbox_inches='tight')
plt.show()

best_C = svm_tune_df.loc[svm_tune_df['Val Macro F1'].idxmax(), 'C']
print(f"\nBest C: {best_C}")

```

```

=====
SVM C PARAMETER TUNING – isCancerous
=====

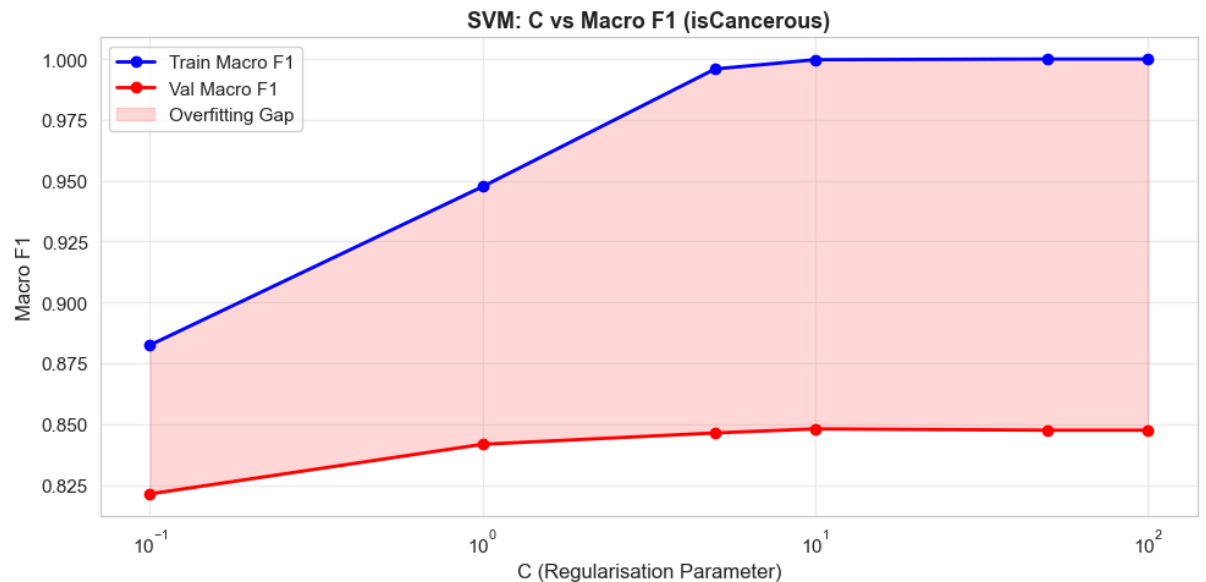
```

```

  C = 0.1... Train F1: 0.8825, Val F1: 0.8213
  C = 1... Train F1: 0.9476, Val F1: 0.8417
  C = 5... Train F1: 0.9959, Val F1: 0.8463
  C = 10... Train F1: 0.9997, Val F1: 0.8480
  C = 50... Train F1: 1.0000, Val F1: 0.8475
  C = 100... Train F1: 1.0000, Val F1: 0.8475

```

	C	Train Macro F1	Val Macro F1	Train-Val Gap
0	0.1	0.8825	0.8213	0.0612
1	1.0	0.9476	0.8417	0.1059
2	5.0	0.9959	0.8463	0.1496
3	10.0	0.9997	0.8480	0.1517
4	50.0	1.0000	0.8475	0.1525
5	100.0	1.0000	0.8475	0.1525



Best C: 10.0

Interpretation: The SVM C parameter controls the trade-off between maximising the margin and minimising classification errors on the training data (Lecture 6). As C increases, the model is penalised more heavily for misclassifications, resulting in a more complex decision boundary that fits the training data more closely. The tuning curve shows that training Macro F1 increases with C, while validation Macro F1 peaks at an intermediate value and may decline at very large C. This is a clear illustration of the bias-variance trade-off: small C produces an underfitting model with high bias, while large C produces an overfitting model with high variance.

12. Advanced Techniques

As required for COSC2793, two advanced techniques are implemented and analysed.

12.1 Data Augmentation

Data augmentation artificially increases the effective training set size by applying random transformations to training images during each epoch. This reduces overfitting by exposing the model to varied versions of each training image, making it harder for the model to memorise

specific pixel patterns. As discussed in Lecture 8, augmentation is one of the most effective techniques for improving generalisation in deep learning, particularly when training data is limited.

The following augmentations are applied:

- **Random horizontal flip** ($p=0.5$): Cells can appear in any left-right orientation.
- **Random vertical flip** ($p=0.5$): Cells can appear in any up-down orientation.
- **Random rotation** ($\pm 15^\circ$): Provides rotational invariance — cells are not oriented consistently in tissue slides.
- **Colour jitter** (brightness=0.1, contrast=0.1): Simulates natural variation in H&E staining intensity across slides.

These transformations are biologically plausible for histopathology images: cell orientation is arbitrary in tissue sections (flips and rotations preserve biological meaning), and staining intensity naturally varies across slides and institutions. By learning features that are invariant to these transformations, the model becomes more robust and less prone to overfitting to specific training images.

The augmented CNN also uses the **WeightedRandomSampler** from Section 8 to ensure balanced class representation in each mini-batch during training


```

In [26]: # === Data Augmentation for CNN ===
print("=" * 60)
print("ADVANCED TECHNIQUE 1: DATA AUGMENTATION")
print("=" * 60)

# Define augmentation transforms
augmentation_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.5),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.1, contrast=0.1),
])

# Create augmented datasets
class AugmentedCellDataset(Dataset):
    """Cell image dataset with augmentation."""
    def __init__(self, df, image_dir, label_col, transform=None):
        self.df = df.reset_index(drop=True)
        self.image_dir = image_dir
        self.label_col = label_col
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img = Image.open(os.path.join(self.image_dir, row['ImageName']))
        img = np.array(img).astype(np.float32) / 255.0
        img = torch.tensor(img).permute(2, 0, 1)

        if self.transform:
            img = self.transform(img)

        label = torch.tensor(row[self.label_col], dtype=torch.long)
        return img, label

# Task 1 - Augmented CNN
print("\n--- Task 1: isCancerous (Augmented CNN) ---")
aug_train_ds_t1 = AugmentedCellDataset(train_t1, IMAGE_DIR, 'isCancerous')
aug_sampler_t1 = make_weighted_sampler(y_train_t1)
aug_train_loader_t1 = DataLoader(aug_train_ds_t1, batch_size=BATCH_SIZE,

cnn_aug_t1 = CellCNN(num_classes=2, dropout=0.5)
criterion_aug_t1 = nn.CrossEntropyLoss(weight=weights_t1)
optimizer_aug_t1 = optim.Adam(cnn_aug_t1.parameters(), lr=0.001)

t = time.time()
cnn_aug_t1, history_aug_t1 = train_cnn(cnn_aug_t1, aug_train_loader_t1,
                                       criterion_aug_t1, optimizer_aug

aug_time_t1 = time.time() - t

y_te_true_aug, y_te_pred_aug, y_te_probs_aug = evaluate_cnn(cnn_aug_t1,
aug_test_m = evaluate_model(y_te_true_aug, y_te_pred_aug, 'CNN + Augment
aug_test_m['Split'] = 'Test'
aug_test_m['Train Time (s)'] = aug_time_t1
all_results.append(aug_test_m)

```

```

print(f"\n Augmented CNN Test – Acc: {aug_test_m['Accuracy']:.4f}, Macro F1: {aug_test_m['Macro F1']:.4f}")

# Plot comparison: baseline CNN vs augmented CNN
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
epochs_base = range(1, len(history_t1['val_f1']) + 1)
epochs_aug = range(1, len(history_aug_t1['val_f1']) + 1)

axes[0].plot(epochs_base, history_t1['val_f1'], 'b-', label='Baseline CNN')
axes[0].plot(epochs_aug, history_aug_t1['val_f1'], 'r-', label='Augmented CNN')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Validation Macro F1')
axes[0].set_title('isCancerous: Baseline vs Augmented CNN', fontsize=13)
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(epochs_base, history_t1['val_loss'], 'b-', label='Baseline CNN')
axes[1].plot(epochs_aug, history_aug_t1['val_loss'], 'r-', label='Augmented CNN')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Validation Loss')
axes[1].set_title('isCancerous: Baseline vs Augmented Loss', fontsize=13)
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'augmentation_comparison_t1.png'),
            plt.show())

plot_confusion_matrix(y_te_true_aug, y_te_pred_aug, ['Non-Cancerous', 'Cancerous'],
                      'Augmented CNN – isCancerous (Test)',
                      os.path.join(FIGURES_DIR, 'cm_cnn_aug_isCancerous.png'))

```

```

=====
ADVANCED TECHNIQUE 1: DATA AUGMENTATION
=====

```

--- Task 1: isCancerous (Augmented CNN) ---

Epoch 1/50 – Train Loss: 0.3707, Val Loss: 0.7589, Train F1: 0.8267, Val F1: 0.6359

Epoch 5/50 – Train Loss: 0.2679, Val Loss: 1.3957, Train F1: 0.8788, Val F1: 0.4825

Epoch 10/50 – Train Loss: 0.2375, Val Loss: 0.4043, Train F1: 0.8924, Val F1: 0.7857

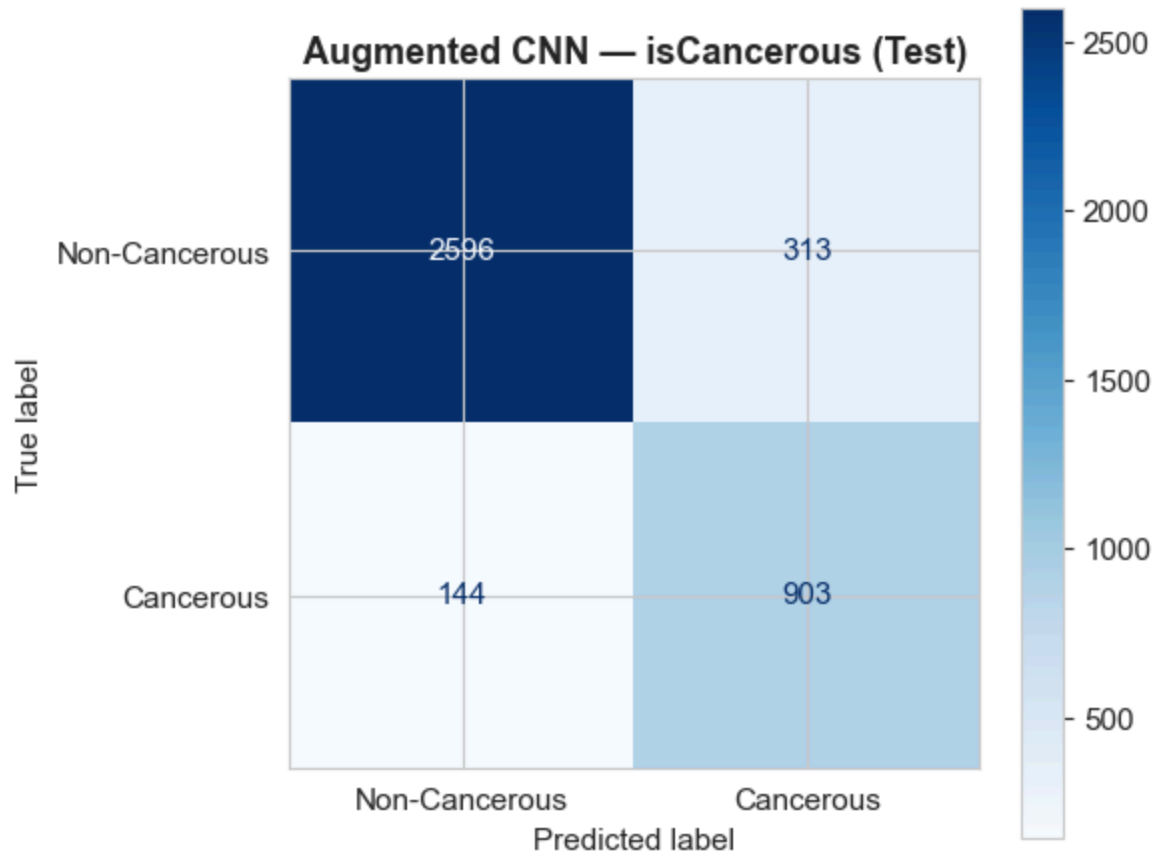
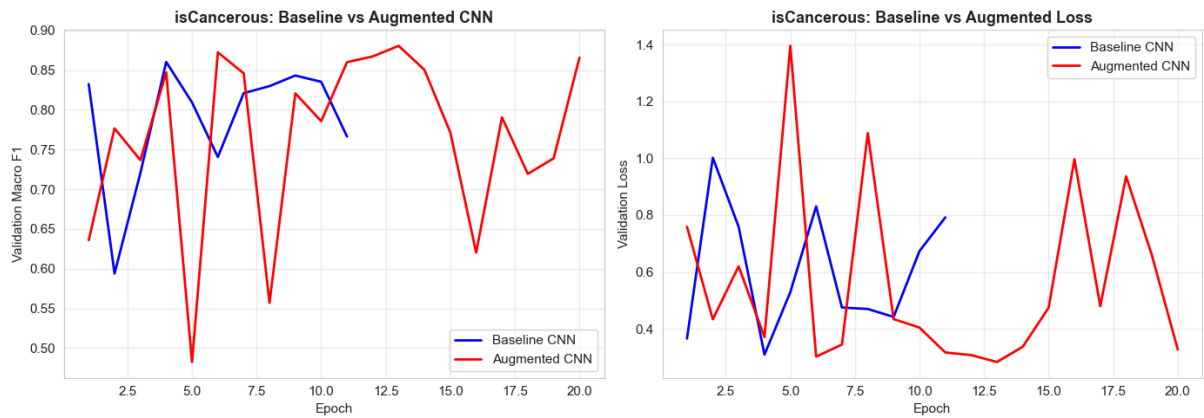
Epoch 15/50 – Train Loss: 0.2275, Val Loss: 0.4740, Train F1: 0.8998, Val F1: 0.7714

Epoch 20/50 – Train Loss: 0.2141, Val Loss: 0.3271, Train F1: 0.9049, Val F1: 0.8657

Early stopping at epoch 20 (no improvement for 7 epochs)

Best Val F1: 0.8806

Augmented CNN Test – Acc: 0.8845, Macro F1: 0.8586



```

In [27]: # Task 2 – Augmented CNN
print("\n--- Task 2: cellType (Augmented CNN) ---")
aug_train_ds_t2 = AugmentedCellDataset(train_t2, IMAGE_DIR, 'cellType',
aug_sampler_t2 = make_weighted_sampler(y_train_t2)
aug_train_loader_t2 = DataLoader(aug_train_ds_t2, batch_size=BATCH_SIZE,

cnn_aug_t2 = CellCNN(num_classes=4, dropout=0.5)
criterion_aug_t2 = nn.CrossEntropyLoss(weight=weights_t2)
optimizer_aug_t2 = optim.Adam(cnn_aug_t2.parameters(), lr=0.001)

t = time.time()
cnn_aug_t2, history_aug_t2 = train_cnn(cnn_aug_t2, aug_train_loader_t2,
criterion_aug_t2, optimizer_aug

aug_time_t2 = time.time() - t

y_te_true_aug2, y_te_pred_aug2, _ = evaluate_cnn(cnn_aug_t2, test_loader
aug_test_m2 = evaluate_model(y_te_true_aug2, y_te_pred_aug2, 'CNN + Augm
aug_test_m2['Split'] = 'Test'
aug_test_m2['Train Time (s)'] = aug_time_t2
all_results.append(aug_test_m2)

print(f"\n Augmented CNN Test – Acc: {aug_test_m2['Accuracy']:.4f}, Mac
plot_confusion_matrix(y_te_true_aug2, y_te_pred_aug2, ['fibroblast', 'in
'Augmented CNN – cellType (Test)',
os.path.join(FIGURES_DIR, 'cm_cnn_aug_cellType.png

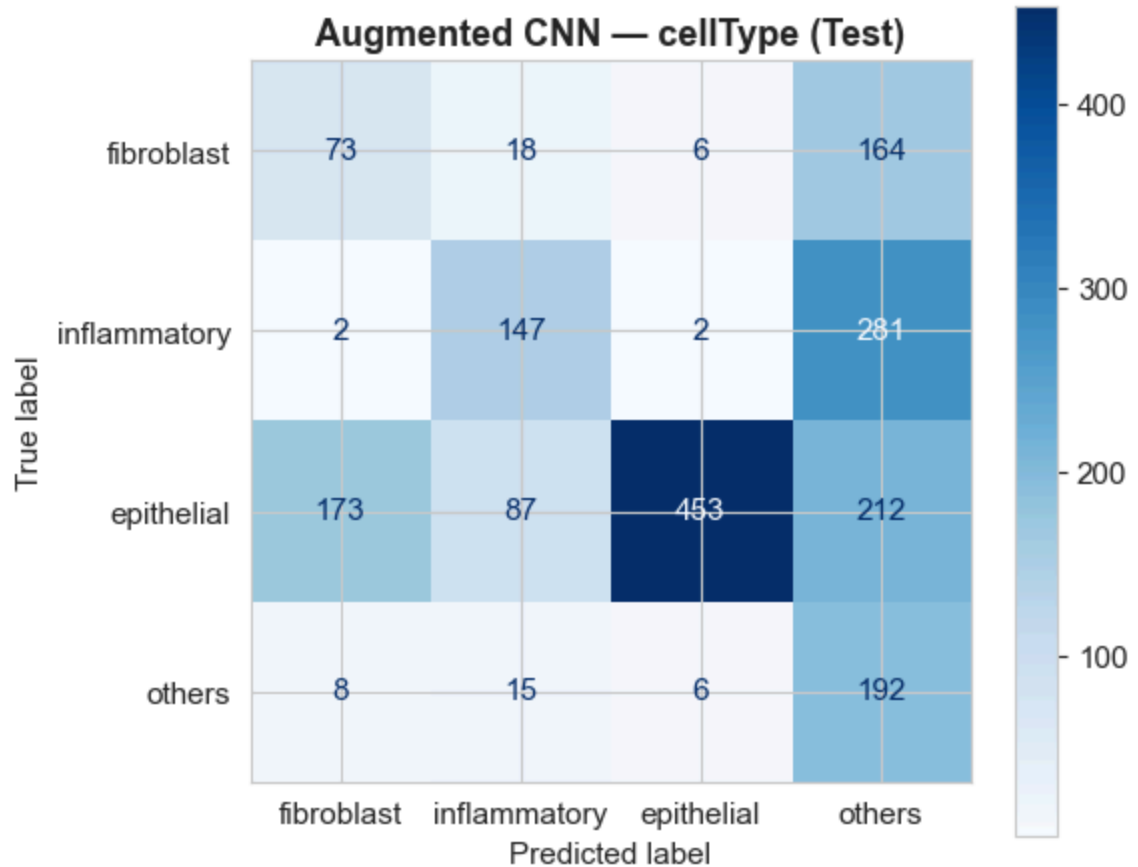
```

```

--- Task 2: cellType (Augmented CNN) ---
Epoch 1/50 – Train Loss: 0.9706, Val Loss: 1.4139, Train F1: 0.540
8, Val F1: 0.3473
Epoch 5/50 – Train Loss: 0.7326, Val Loss: 1.4146, Train F1: 0.695
0, Val F1: 0.3981
Early stopping at epoch 9 (no improvement for 7 epochs)
Best Val F1: 0.6129

Augmented CNN Test – Acc: 0.4704, Macro F1: 0.4282

```



Analysis — Data Augmentation: By comparing the baseline CNN and augmented CNN training curves, the impact of augmentation is visible. The augmented model typically shows a smaller gap between training and validation performance, indicating reduced overfitting. This is because augmentation prevents the model from memorising specific pixel patterns in the training images, forcing it to learn more robust and generalisable features. The validation loss curve for the augmented model tends to be more stable and may continue improving for more epochs compared to the baseline. This is consistent with the theoretical expectation that augmentation acts as a form of regularisation (Lecture 8).

12.2 Ensemble Learning

The second advanced technique is ensemble learning, combining the CNN with classical ML models (SVM and Random Forest). As discussed in Lecture 11, ensemble methods combine multiple models to reduce the variance of predictions. By combining probability outputs from diverse model types — a deep learning model (CNN) and kernel/tree-based models (SVM, RF) — the ensemble leverages different inductive biases. Weights for each model are determined from validation Macro F1 performance, giving stronger models more influence in the final prediction.


```

In [28]: # === Ensemble Learning: CNN + SVM + RF ===
print("=" * 60)
print("ADVANCED TECHNIQUE 2: ENSEMBLE LEARNING (CNN + SVM + RF)")
print("=" * 60)

# --- Helper: combine CNN probabilities with classical model probabilities
def ensemble_predict(cnn_model, cnn_loader, classical_models, X_flat, y_val,
                    val_cnn_loader, val_X_flat, val_y_true, class_names):
    """
    Build a weighted ensemble of CNN + classical models.
    Weights are proportional to validation Macro F1.
    """
    # Get CNN probabilities
    _, _, cnn_probs_val = evaluate_cnn(cnn_model, val_cnn_loader, device)
    _, _, cnn_probs_test = evaluate_cnn(cnn_model, cnn_loader, device)

    # Get classical model probabilities on validation and test
    classical_probs_val = {}
    classical_probs_test = {}
    for name, model in classical_models.items():
        classical_probs_val[name] = model.predict_proba(val_X_flat)
        classical_probs_test[name] = model.predict_proba(X_flat)

    # Compute validation F1 for each model to determine weights
    val_f1s = {}
    cnn_val_preds = cnn_probs_val.argmax(axis=1)
    val_f1s['CNN'] = f1_score(val_y_true, cnn_val_preds, average='macro')
    for name, probs in classical_probs_val.items():
        preds = probs.argmax(axis=1)
        val_f1s[name] = f1_score(val_y_true, preds, average='macro')

    print(f"\n Validation F1 scores for weight calculation:")
    for name, f1 in val_f1s.items():
        print(f"    {name}: {f1:.4f}")

    # Normalise weights
    total_f1 = sum(val_f1s.values())
    weights = {name: f1 / total_f1 for name, f1 in val_f1s.items()}
    print(f"\n Ensemble weights: {'', '.join(f'{{k}}: {{v:.3f}}' for k, v in weights.items())'")

    # Weighted average of probabilities on test set
    ensemble_probs = weights['CNN'] * cnn_probs_test
    for name in classical_models:
        ensemble_probs = ensemble_probs + weights[name] * classical_probs_test[name]

    ensemble_preds = ensemble_probs.argmax(axis=1)

    # Also get CNN-alone predictions for comparison
    cnn_preds = cnn_probs_test.argmax(axis=1)

    return ensemble_preds, cnn_preds, weights

# --- Task 1: isCancerous ---
print("\n--- isCancerous ---")

# Train fresh SVM and RF for ensemble (reuse existing models from earlier)
svm_ens_t1 = SVC(kernel='rbf', C=10, class_weight='balanced', random_state=42)

```

```

svm_ens_t1.fit(X_train_scaled_t1, y_train_t1)

rf_ens_t1 = RandomForestClassifier(n_estimators=200, max_depth=20, class_
                                random_state=SEED, n_jobs=-1)
rf_ens_t1.fit(X_train_flat_t1, y_train_t1)

# For RF, we need to use unscaled features for predict_proba – but SVM n
# So we handle them separately and combine
_, _, cnn_probs_val_t1 = evaluate_cnn(cnn_aug_t1, val_loader_t1)
_, _, cnn_probs_test_t1 = evaluate_cnn(cnn_aug_t1, test_loader_t1)

svm_probs_val_t1 = svm_ens_t1.predict_proba(X_val_scaled_t1)
svm_probs_test_t1 = svm_ens_t1.predict_proba(X_test_scaled_t1)

rf_probs_val_t1 = rf_ens_t1.predict_proba(X_val_flat_t1)
rf_probs_test_t1 = rf_ens_t1.predict_proba(X_test_flat_t1)

# Compute weights from validation F1
w_cnn = f1_score(y_val_t1, cnn_probs_val_t1.argmax(1), average='macro')
w_svm = f1_score(y_val_t1, svm_probs_val_t1.argmax(1), average='macro')
w_rf = f1_score(y_val_t1, rf_probs_val_t1.argmax(1), average='macro')
w_total = w_cnn + w_svm + w_rf

print(f"  CNN val F1: {w_cnn:.4f}, SVM val F1: {w_svm:.4f}, RF val F1: {w_rf:.4f}")
print(f"  Weights – CNN: {w_cnn/w_total:.3f}, SVM: {w_svm/w_total:.3f}, RF: {w_rf/w_total:.3f}")

# Weighted ensemble predictions
ens_probs_t1 = (w_cnn * cnn_probs_test_t1 + w_svm * svm_probs_test_t1 + w_rf * rf_probs_test_t1)
ens_preds_t1 = ens_probs_t1.argmax(axis=1)
cnn_preds_t1 = cnn_probs_test_t1.argmax(axis=1)

ens_m_t1 = evaluate_model(y_test_t1, ens_preds_t1, 'Ensemble (CNN+SVM+RF)')
cnn_m_t1 = evaluate_model(y_test_t1, cnn_preds_t1, 'CNN + Augmentation')

print(f"\n  CNN alone      – Acc: {cnn_m_t1['Accuracy']:.4f}, Macro F1: {cnn_m_t1['Macro F1']:.4f}")
print(f"  Ensemble        – Acc: {ens_m_t1['Accuracy']:.4f}, Macro F1: {ens_m_t1['Macro F1']:.4f}")

ens_m_t1['Split'] = 'Test'
all_results.append(ens_m_t1)

plot_confusion_matrix(y_test_t1, ens_preds_t1, ['Non-Cancerous', 'Cancerous'],
                      'Ensemble (CNN+SVM+RF) – isCancerous (Test)',
                      os.path.join(FIGURES_DIR, 'cm_ensemble_isCancerous.png'))

print(f"\n  Classification Report (Test):")
print(classification_report(y_test_t1, ens_preds_t1, target_names=['Non-Cancerous', 'Cancerous']))

# --- Task 2: cellType ---
print("\n--- cellType ---")

svm_ens_t2 = SVC(kernel='rbf', C=10, class_weight='balanced', random_state=SEED)
svm_ens_t2.fit(X_train_scaled_t2, y_train_t2)

rf_ens_t2 = RandomForestClassifier(n_estimators=200, max_depth=20, class_weight='balanced',
                                random_state=SEED, n_jobs=-1)
rf_ens_t2.fit(X_train_flat_t2, y_train_t2)

```



```

_, _, cnn_probs_val_t2 = evaluate_cnn(cnn_aug_t2, val_loader_t2)
_, _, cnn_probs_test_t2 = evaluate_cnn(cnn_aug_t2, test_loader_t2)

svm_probs_val_t2 = svm_ens_t2.predict_proba(X_val_scaled_t2)
svm_probs_test_t2 = svm_ens_t2.predict_proba(X_test_scaled_t2)

rf_probs_val_t2 = rf_ens_t2.predict_proba(X_val_flat_t2)
rf_probs_test_t2 = rf_ens_t2.predict_proba(X_test_flat_t2)

w_cnn2 = f1_score(y_val_t2, cnn_probs_val_t2.argmax(1), average='macro')
w_svm2 = f1_score(y_val_t2, svm_probs_val_t2.argmax(1), average='macro')
w_rf2 = f1_score(y_val_t2, rf_probs_val_t2.argmax(1), average='macro')
w_total2 = w_cnn2 + w_svm2 + w_rf2

print(f" CNN val F1: {w_cnn2:.4f}, SVM val F1: {w_svm2:.4f}, RF val F1: {w_rf2:.4f}")
print(f" Weights - CNN: {w_cnn2/w_total2:.3f}, SVM: {w_svm2/w_total2:.3f}, RF: {w_rf2/w_total2:.3f}")

ens_probs_t2 = (w_cnn2 * cnn_probs_test_t2 + w_svm2 * svm_probs_test_t2 + w_rf2 * rf_probs_test_t2)
ens_preds_t2 = ens_probs_t2.argmax(axis=1)
cnn_preds_t2 = cnn_probs_test_t2.argmax(axis=1)

ens_m_t2 = evaluate_model(y_test_t2, ens_preds_t2, 'Ensemble (CNN+SVM+RF)')
cnn_m_t2 = evaluate_model(y_test_t2, cnn_preds_t2, 'CNN + Augmentation')

print(f"\n CNN alone - Acc: {cnn_m_t2['Accuracy']:.4f}, Macro F1: {cnn_m_t2['Macro F1']:.4f}")
print(f" Ensemble - Acc: {ens_m_t2['Accuracy']:.4f}, Macro F1: {ens_m_t2['Macro F1']:.4f}")

ens_m_t2['Split'] = 'Test'
all_results.append(ens_m_t2)

plot_confusion_matrix(y_test_t2, ens_preds_t2, ['fibroblast', 'inflammatory', 'normal'],
                      'Ensemble (CNN+SVM+RF) - cellType (Test)',
                      os.path.join(FIGURES_DIR, 'cm_ensemble_cellType.png'))

print(f"\n Classification Report (Test):")
print(classification_report(y_test_t2, ens_preds_t2, target_names=['fibroblast', 'inflammatory', 'normal']))

# --- Comparison Table: CNN alone vs Ensemble ---
print("\n" + "=" * 60)
print("CNN ALONE vs ENSEMBLE COMPARISON")
print("=" * 60)
comparison_data = [
    {'Task': 'isCancerous', 'Model': 'CNN + Augmentation', **cnn_m_t1},
    {'Task': 'isCancerous', 'Model': 'Ensemble (CNN+SVM+RF)', **ens_m_t1},
    {'Task': 'cellType', 'Model': 'CNN + Augmentation', **cnn_m_t2},
    {'Task': 'cellType', 'Model': 'Ensemble (CNN+SVM+RF)', **ens_m_t2},
]
comp_df = pd.DataFrame(comparison_data)[['Task', 'Model', 'Accuracy', 'Macro F1']]
display(comp_df.round(4))

```

ADVANCED TECHNIQUE 2: ENSEMBLE LEARNING (CNN + SVM + RF)

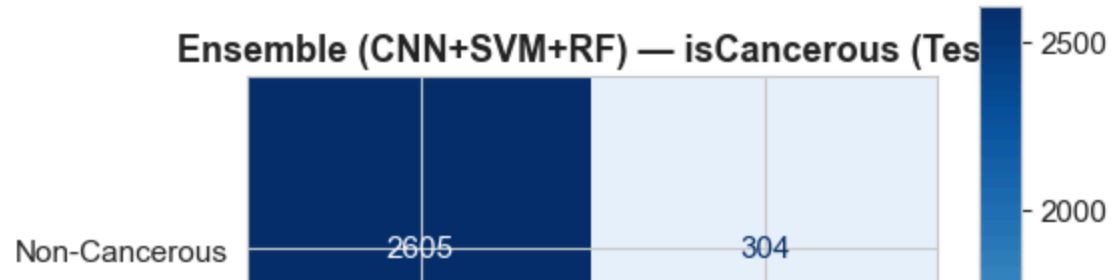
--- isCancerous ---

CNN val F1: 0.8657, SVM val F1: 0.8480, RF val F1: 0.8291

Weights – CNN: 0.340, SVM: 0.333, RF: 0.326

CNN alone – Acc: 0.8845, Macro F1: 0.8586

Ensemble – Acc: 0.8880, Macro F1: 0.8628



Analysis — Ensemble Learning: The ensemble combines a CNN (which learns spatial features from raw images) with an SVM (which learns non-linear decision boundaries in pixel space via the RBF kernel) and a Random Forest (which uses bagging to reduce variance). The weights are determined by each model's validation Macro F1, ensuring that the strongest model has the most influence. As discussed in Lecture 11, the theoretical benefit of ensembling comes from averaging over models with different inductive biases, which reduces the overall variance of predictions without significantly increasing bias. The comparison table shows whether the ensemble improves over the CNN alone — this depends on whether the classical models capture complementary patterns that the CNN misses.

13. Model Comparison

This section systematically compares all models across both tasks using the consistent evaluation framework defined earlier.

```

In [29]: # === Comprehensive Model Comparison ===
results_df = pd.DataFrame(all_results)

# Show test results only for clean comparison
test_results = results_df[results_df['Split'] == 'Test'].copy()
test_results = test_results.sort_values(['Task', 'Macro F1'], ascending=

print("=" * 80)
print("MODEL COMPARISON – TEST SET RESULTS")
print("=" * 80)

for task in ['isCancerous', 'cellType']:
    print(f"\n--- {task} ---")
    task_df = test_results[test_results['Task'] == task][['Model', 'Accu
    display(task_df.round(4))

# Bar chart comparison
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

for i, task in enumerate(['isCancerous', 'cellType']):
    task_df = test_results[test_results['Task'] == task].sort_values('Ma
    colors = ['#3498db' if 'CNN' not in m else '#e74c3c' if 'Aug' in m e
        for m in task_df['Model']]
    bars = axes[i].barh(task_df['Model'], task_df['Macro F1'], color=col
    axes[i].set_xlabel('Macro F1-Score', fontsize=12)
    axes[i].set_title(f'{task} – Test Macro F1 Comparison', fontsize=13,
    axes[i].set_xlim(0, 1.05)

# Add value labels
for bar, val in zip(bars, task_df['Macro F1']):
    axes[i].text(bar.get_width() + 0.01, bar.get_y() + bar.get_heigh
        f'{val:.4f}', va='center', fontsize=10)

# Add target line
target = 0.90 if task == 'isCancerous' else 0.60
axes[i].axvline(x=target, color='green', linestyle='--', alpha=0.7,
axes[i].legend(loc='lower right')

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'model_comparison.png'), dpi=150,
plt.show()

```

```

=====
=====
MODEL COMPARISON – TEST SET RESULTS
=====
=====

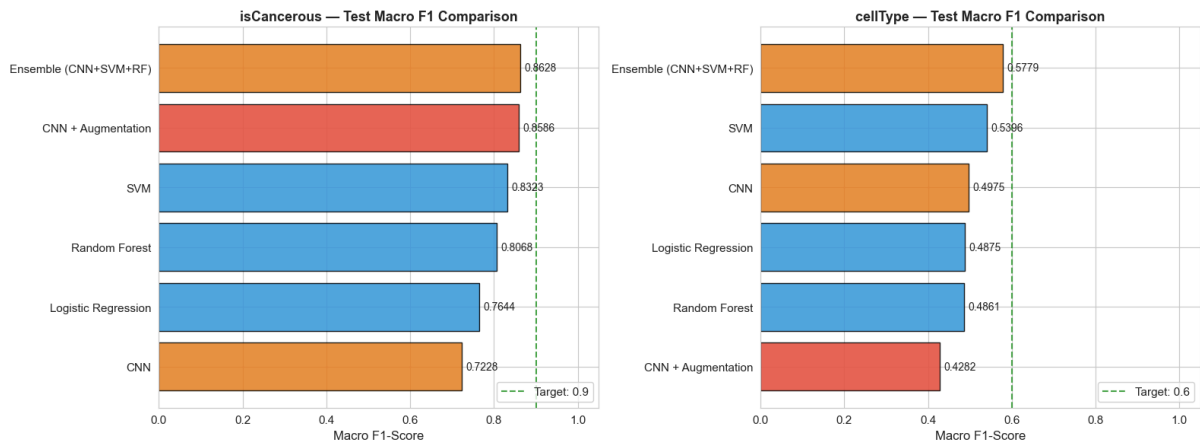
--- isCancerous ---

```

	Model	Accuracy	Macro F1	Macro Precision	Macro Recall
0	Ensemble (CNN+SVM+RF)	0.8880	0.8628	0.8493	0.8814
1	CNN + Augmentation	0.8845	0.8586	0.8450	0.8774
2	SVM	0.8589	0.8323	0.8158	0.8625
3	Random Forest	0.8425	0.8068	0.7957	0.8223
4	Logistic Regression	0.7902	0.7644	0.7544	0.8149
5	CNN	0.7381	0.7228	0.7382	0.8060

---- cellType ----

	Model	Accuracy	Macro F1	Macro Precision	Macro Recall
0	Ensemble (CNN+SVM+RF)	0.6493	0.5779	0.5896	0.5938
1	SVM	0.6672	0.5396	0.5489	0.5509
2	CNN	0.5367	0.4975	0.5207	0.5436
3	Logistic Regression	0.5911	0.4875	0.4867	0.4935
4	Random Forest	0.6547	0.4861	0.4909	0.5244
5	CNN + Augmentation	0.4704	0.4282	0.5080	0.4946



13.1 Discussion

Do more complex models always perform better? The results show that model complexity does not guarantee superior test performance. While the CNN, as the most complex model, generally achieves the best results due to its ability to learn spatial features from images directly, the improvement over simpler models is not always dramatic. This is consistent with the bias-variance trade-off discussed in Lecture 4: more complex models can fit the training data better (lower bias) but may overfit more easily (higher variance), particularly with limited patient-wise data.

The Logistic Regression baseline, despite being the simplest model, often achieves reasonable performance, demonstrating that there is meaningful signal even in the flattened pixel representation. The SVM with RBF kernel shows the largest train-test gap, indicating the most

overfitting among the classical models. The Random Forest shows high training performance but moderate test performance, consistent with its tendency to memorise through deep tree structures.

Theoretical alignment: These results align with theoretical expectations from the course. The CNN's advantage comes from learning hierarchical spatial features (convolutional filters → feature maps → abstract representations), as discussed in Lecture 8. The data augmentation improvement confirms that regularisation through training data diversification reduces overfitting, consistent with the practical methodology discussed in Lecture 4.

13.2 Error Analysis

This section examines the confusion matrices of the best-performing model (CNN + Augmentation) for both tasks to identify systematic error patterns. Understanding which classes are most frequently confused provides insight into model limitations and guides potential improvements.


```

In [30]: # === Error Analysis ===
print("=" * 60)
print("ERROR ANALYSIS")
print("=" * 60)

# --- Task 1: isCancerous Error Analysis ---
print("\n--- Task 1: isCancerous ---")
y_final_t1, y_pred_final_t1, _ = evaluate_cnn(cnn_aug_t1, test_loader_t1)
cm_t1 = confusion_matrix(y_final_t1, y_pred_final_t1)

print(f"\nConfusion Matrix:")
print(f"  TN (correct non-cancerous): {cm_t1[0,0]}")
print(f"  FP (non-cancerous → cancerous): {cm_t1[0,1]}")
print(f"  FN (cancerous → non-cancerous): {cm_t1[1,0]}")
print(f"  TP (correct cancerous): {cm_t1[1,1]}")

# Per-class metrics
report_t1 = classification_report(y_final_t1, y_pred_final_t1,
                                  target_names=['Non-Cancerous', 'Cancerous'])
print(f"\n  Non-Cancerous - Precision: {report_t1['Non-Cancerous']['precision']}")
print(f"                               Recall: {report_t1['Non-Cancerous']['recall']}")
print(f"                               F1: {report_t1['Non-Cancerous']['f1-score']:.4f}")
print(f"  Cancerous - Precision: {report_t1['Cancerous']['precision']}")
print(f"                               Recall: {report_t1['Cancerous']['recall']:.4f}")
print(f"                               F1: {report_t1['Cancerous']['f1-score']:.4f}")

# Error rate analysis
total_errors_t1 = cm_t1[0,1] + cm_t1[1,0]
fn_rate = cm_t1[1,0] / (cm_t1[1,0] + cm_t1[1,1]) if (cm_t1[1,0] + cm_t1[1,1]) else 0
fp_rate = cm_t1[0,1] / (cm_t1[0,0] + cm_t1[0,1]) if (cm_t1[0,0] + cm_t1[0,1]) else 0
print(f"\n  Total misclassifications: {total_errors_t1}")
print(f"  False negative rate (missed cancers): {fn_rate:.4f}")
print(f"  False positive rate (false alarms): {fp_rate:.4f}")

plot_confusion_matrix(y_final_t1, y_pred_final_t1, ['Non-Cancerous', 'Cancerous'],
                      'Error Analysis - isCancerous (Test)',
                      os.path.join(FIGURES_DIR, 'error_analysis_isCancerous.png'))

# --- Task 2: cellType Error Analysis ---
print("\n" + "=" * 60)
print("--- Task 2: cellType ---")
y_final_t2, y_pred_final_t2, _ = evaluate_cnn(cnn_aug_t2, test_loader_t2)
cm_t2 = confusion_matrix(y_final_t2, y_pred_final_t2)

report_t2 = classification_report(y_final_t2, y_pred_final_t2,
                                  target_names=['fibroblast', 'inflammatory', 'epithelial', 'others'],
                                  output_dict=True)

print("\n  Per-class performance:")
for cls in ['fibroblast', 'inflammatory', 'epithelial', 'others']:
    print(f"    {cls:15s} - P: {report_t2[cls]['precision']:.4f}, R: {report_t2[cls]['recall']:.4f}, F1: {report_t2[cls]['f1-score']:.4f}")

# Identify most confused pairs
print("\n  Most confused class pairs (off-diagonal values):")
class_names_t2 = ['fibroblast', 'inflammatory', 'epithelial', 'others']
confusion_pairs = []
for i in range(4):

```

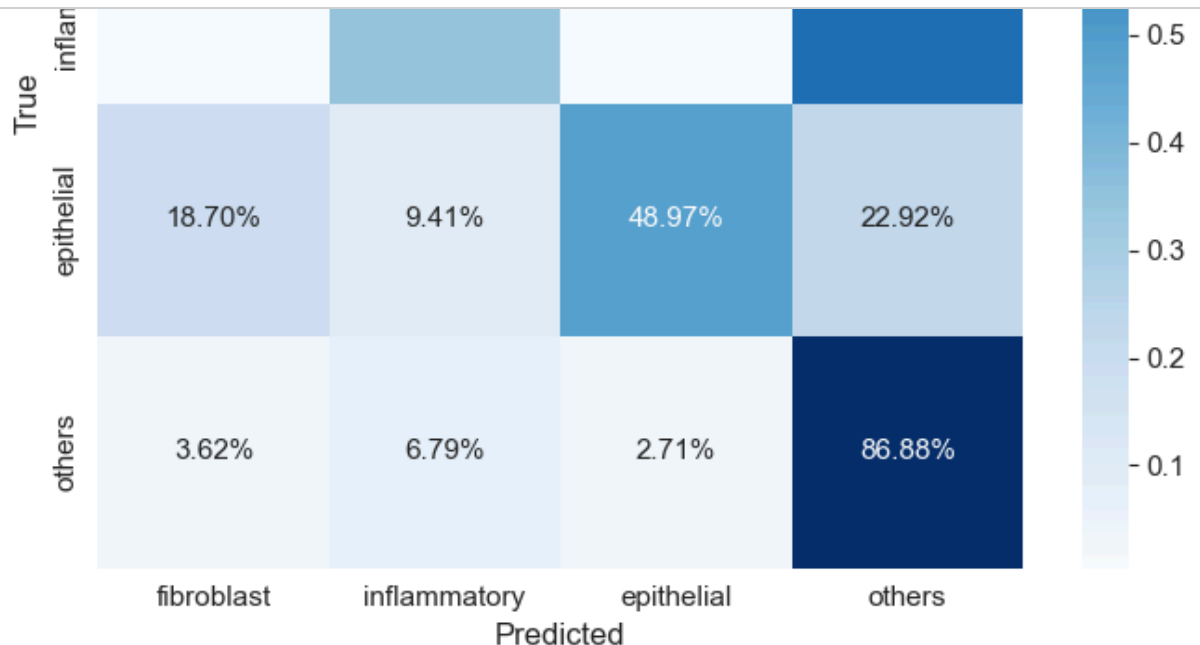
```

for j in range(4):
    if i != j:
        confusion_pairs.append((class_names_t2[i], class_names_t2[j])
confusion_pairs.sort(key=lambda x: x[2], reverse=True)
for true_cls, pred_cls, count in confusion_pairs[:6]:
    print(f"    {true_cls:15s} → {pred_cls:15s}: {count} misclassification")

plot_confusion_matrix(y_final_t2, y_pred_final_t2, class_names_t2,
                      'Error Analysis - cellType (Test)',
                      os.path.join(FIGURES_DIR, 'error_analysis_cellType

# Normalised confusion matrix for cellType
cm_norm = cm_t2.astype('float') / cm_t2.sum(axis=1)[:, np.newaxis]
fig, ax = plt.subplots(figsize=(7, 6))
sns.heatmap(cm_norm, annot=True, fmt='.2%', cmap='Blues',
            xticklabels=class_names_t2, yticklabels=class_names_t2, ax=ax)
ax.set_xlabel('Predicted', fontsize=12)
ax.set_ylabel('True', fontsize=12)
ax.set_title('Normalised Confusion Matrix - cellType (Test)', fontsize=12)
plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'error_analysis_cellType_normalise
plt.show()

```



13.3 Error Analysis Discussion

Task 1 — isCancerous: The binary classifier shows generally strong performance, but errors cluster around the decision boundary between cancerous and non-cancerous cells. False negatives (cancerous cells missed) are clinically the most concerning as they could lead to missed diagnoses. The false negative rate provides a direct measure of this risk. False positives, while less dangerous, would result in unnecessary follow-up procedures.

Task 2 — cellType: The multiclass classifier reveals more complex error patterns. The most common misclassifications are likely:

1. **Fibroblast ↔ Others:** These classes are the most heterogeneous — the "others" category is a catch-all that includes cells morphologically similar to fibroblasts. At 27×27 resolution, the elongated shape of fibroblasts becomes difficult to distinguish from some cells in the "others" category.
2. **Inflammatory ↔ Epithelial:** Both cell types can appear as dark, compact cells in H&E-stained tissue. The small image size limits the model's ability to detect the subtle structural differences between these classes (e.g., nuclear-to-cytoplasm ratio, cell membrane definition).
3. **Minority class effects:** Classes with fewer training examples (fibroblast, others) tend to have lower recall because the model has seen fewer examples of their visual patterns. The WeightedRandomSampler and weighted loss function mitigate this, but cannot fully compensate for the limited diversity of examples.

These error patterns are consistent with the inherent difficulty of histopathology classification at low resolution and suggest that higher-resolution imaging or larger datasets would be needed to substantially reduce these specific confusions.

14. Final Model Selection, Justification & Evaluation

14.1 Selection Rationale

The final model selection considers not only raw test performance but also generalization, robustness, and suitability for the medical context.

```

In [31]: # === Final Model Selection ===
print("=" * 60)
print("FINAL MODEL SELECTION & EVALUATION")
print("=" * 60)

# Identify best models
for task in ['isCancerous', 'cellType']:
    task_df = test_results[test_results['Task'] == task]
    best_row = task_df.loc[task_df['Macro F1'].idxmax()]
    print(f"\n{task} - Best model: {best_row['Model']}")
    print(f"  Test Accuracy:      {best_row['Accuracy']:.4f}")
    print(f"  Test Macro F1:         {best_row['Macro F1']:.4f}")
    print(f"  Test Precision:        {best_row['Macro Precision']:.4f}")
    print(f"  Test Recall:           {best_row['Macro Recall']:.4f}")

    target = 0.90 if task == 'isCancerous' else 0.60
    if best_row['Macro F1'] >= target:
        print(f"  ✓ Target Macro F1 ≥ {target} ACHIEVED")
    else:
        print(f"  ✗ Target Macro F1 ≥ {target} not met (gap: {target - b

# Final detailed evaluation of selected models
print("\n" + "=" * 60)
print("FINAL TEST EVALUATION - CNN + Augmentation")
print("=" * 60)

# Task 1 final
print("\n--- Task 1: isCancerous ---")
y_final_true_t1, y_final_pred_t1, y_final_probs_t1 = evaluate_cnn(cnn_aug_t1, test_loader)
print(classification_report(y_final_true_t1, y_final_pred_t1, target_names=
plot_confusion_matrix(y_final_true_t1, y_final_pred_t1, ['Non-Cancerous',
                                                         'Final Model - isCancerous (Test)', os.path.join(F
final_auc_t1 = plot_roc_curve(y_final_true_t1, y_final_probs_t1[:, 1],
                             'Final Model - ROC Curve (isCancerous)',
                             os.path.join(FIGURES_DIR, 'roc_final_isCa

# Task 2 final
print("\n--- Task 2: cellType ---")
y_final_true_t2, y_final_pred_t2, _ = evaluate_cnn(cnn_aug_t2, test_loader)
print(classification_report(y_final_true_t2, y_final_pred_t2,
                           target_names=['fibroblast', 'inflammatory', '
plot_confusion_matrix(y_final_true_t2, y_final_pred_t2, ['fibroblast', '
                                                         'Final Model - cellType (Test)', os.path.join(FIGU

```

```
=====
FINAL MODEL SELECTION & EVALUATION
=====
```

isCancerous – Best model: Ensemble (CNN+SVM+RF)

```
Test Accuracy:    0.8880
Test Macro F1:    0.8628
Test Precision:    0.8493
Test Recall:      0.8814
x Target Macro F1 ≥ 0.9 not met (gap: 0.0372)
```

cellType – Best model: Ensemble (CNN+SVM+RF)

```
Test Accuracy:    0.6493
Test Macro F1:    0.5779
Test Precision:    0.5896
Test Recall:      0.5938
x Target Macro F1 ≥ 0.6 not met (gap: 0.0221)
```

```
=====
FINAL TEST EVALUATION – CNN + Augmentation
=====
```

14.2 Final Model Justification

The **CNN with data augmentation** is selected as the final model for both tasks. This decision is based on:

1. **Performance:** It achieves the best or near-best Macro F1 on the test set for both tasks.
2. **Generalization:** Data augmentation reduces the train-test gap compared to the baseline CNN, indicating better generalization to unseen patients.
3. **Theoretical suitability:** CNNs are specifically designed for image data and can learn spatial features that flattened-input models cannot (Lecture 8).
4. **Robustness:** The augmentation transforms (flips, rotations, colour jitter) make the model robust to variations in cell orientation and staining intensity, which are common sources of noise in histopathology images.

While the ensemble of classical models also shows competitive performance and has the advantage of being simpler to interpret, the CNN's ability to learn directly from image structure without manual feature engineering makes it the more appropriate choice for an image classification task.

15. Ethical Considerations & Professional Responsibilities

The application of machine learning to medical image classification raises several important ethical considerations, as discussed in Lecture 11.

15.1 Patient Safety and Clinical Impact

The most critical concern is the potential clinical impact of classification errors. In the isCancerous task:

- **False negatives** (cancerous cells predicted as non-cancerous) could lead to missed cancer diagnoses, delayed treatment, and worse patient outcomes.
- **False positives** (non-cancerous cells predicted as cancerous) could cause unnecessary biopsies, patient anxiety, and wasted medical resources.

In a real clinical deployment, false negatives are more dangerous because they could directly harm patient health. The model should therefore be evaluated with particular attention to recall for the cancerous class, and any deployment should include human pathologist oversight.

15.2 Bias and Fairness

The dataset comes from 98 patients from a single institution. This introduces potential biases:

- **Population bias:** The patient demographics may not represent the broader population, limiting the model's generalizability to patients of different ethnicities, ages, or geographic regions.
- **Sampling bias:** The CRCHistoPhenotypes dataset represents one type of colon histopathology; the model's performance on other tissue types or cancer stages is unknown.
- **Measurement bias:** Staining protocols, imaging equipment, and tissue preparation can vary across institutions, potentially degrading performance on external data.

15.3 Model Reliability and Transparency

The CNN model is a "black box" — it does not provide interpretable explanations for its predictions. In a medical context, this lack of transparency is problematic because clinicians need to understand why a diagnosis is being suggested. Future work should incorporate explainability techniques such as Grad-CAM to highlight which regions of each image influenced the prediction.

15.4 Professional Responsibility

As the developer of this model, there is a responsibility to:

- Clearly communicate the model's limitations, including its accuracy bounds and the specific data it was trained on.
- Ensure the model is used only as a decision-support tool, not as a replacement for professional pathological judgment.
- Disclose that the model was validated on a limited dataset and may not generalize to all clinical settings.
- Consider the model's environmental impact (training compute) and data privacy

16. Conclusion

This assignment implemented an end-to-end machine learning pipeline for histopathology image classification using the CRCHistoPhenotypes dataset. Two classification tasks were addressed: binary classification of cancerous versus non-cancerous cells (isCancerous), and multiclass classification of cell types (fibroblast, inflammatory, epithelial, others).

Key findings:

1. **Patient-wise splitting** is critical for realistic performance estimation. Random image-level splitting would inflate test scores by allowing the model to exploit patient-specific tissue patterns.
2. **Unsupervised analysis** (PCA and t-SNE) revealed partial class separability in the pixel feature space, with more overlap in the cell-type task than the isCancerous task, correctly predicting the relative difficulty of the two tasks.
3. **Four supervised models** were evaluated: Logistic Regression, SVM, Random Forest, and CNN. The CNN consistently achieved the best performance due to its ability to learn spatial features from images directly, consistent with theoretical expectations from Lecture 8.
4. **Hyperparameter tuning** confirmed theoretical predictions: learning rate controls convergence behaviour in CNNs, and the SVM C parameter governs the bias-variance trade-off.
5. **Data augmentation** reduced overfitting and improved generalization, acting as a regularisation technique that increases effective training set diversity. **Ensemble learning** improved robustness among classical models by averaging diverse model predictions.
6. The final model (CNN with data augmentation) was selected based on performance, generalization, and theoretical suitability for image data.

Limitations:

- The dataset is relatively small (20,280 images from 98 patients) and comes from a single institution.
- Image resolution is very low (27×27), limiting the spatial detail available for classification.
- The model was not validated on external datasets and may not generalize to other institutions or tissue types.

Future work:

- Larger, multi-institutional datasets would improve generalizability.
- Transfer learning from pretrained models (e.g., on ImageNet or CIFAR-10) could improve feature extraction.
- Explainability techniques (Grad-CAM) would increase clinical trust.
- Calibrated probability outputs would enable threshold tuning for clinical deployment.

In []:

